selected. The documents in the right frames are short descriptions of the people.

2.15 Create, test, and validate an XHTML document that has two frames in one column. The top frame, which must be 20 percent of the column, must have at least four links to other documents; the bottom frame will display those documents. The links must be names of cars; the documents must be at least five-line descriptions of the cars.

2.16 Create, test, and validate an XHTML document that has a form with the following controls:

a. A text box to collect the user's name

b. Four checkboxes, one each for the following items:

i. Four 100-watt light bulbs for $2.39

ii. Eight 100-watt light bulbs for $4.29

iii. Four 100-watt, long-life light bulbs for $3.95

iv. Eight 100-watt, long-life light bulbs for $7.49

c. A collection of three radio buttons that are labeled as follows:

i. Visa

ii. Mastercard

iii. Discover

2.17 Create, test, and validate an XHTML document that has five frames. There must be two rows of frames, the first with three frames and the other with two frames. The frames in the first row must have equal width. The left frame in the second row must be 55 percent of the width of the display. Each of the frames in the top row must display a document that has a form. The left top frame must have two text boxes, each 30 characters wide, labeled Name and Address. The middle top frame must have five radio buttons with color name labels. The right top frame must have four checkboxes, labeled with four kinds of car equipment such as a CD player and air conditioning. The two bottom frames must have images of two different cars. The top row of frames must use 20 percent of the height of the display.

CHAPTER

# 3

# Cascading Style Sheets


3.1 Introduction
3.2 Levels of Style Sheets
3.3 Style Specification Formats
3.4 Selector Forms
3.5 Property Value Forms
3.6 Font Properties
3.7 List Properties
3.8 Color
3.9 Alignment of Text
3.10 The Box Model
3.11 Background Images
3.12 The `<span>` and `<div>` Tags
3.13 Conflict Resolution
*Summary • Review Questions • Exercises*


**This chapter introduces the concept of a style sheet and explains how style sheets fit into the philosophy of XHTML and the structure of XHTML documents.** Next, the three levels of style sheets and the format of style specifications are introduced. Then, the many varieties of property value forms are described. Next, specific properties for fonts and lists are introduced and illustrated. A discussion of the properties for specifying colors, background images, and text alignment follows. The box model of document elements is then dis-

cussed, along with borders and the associated padding and margin properties. The chapter's next section describes two tags, `<span>` and `<div>`, that are used to delimit the scope of style-sheet specifications. These tags are used in full examples in Chapter 6, "Dynamic Documents with JavaScript." Finally, the last section of the chapter provides an overview of the resolution process for conflicting style specifications.

## 3.1 Introduction

We have said that XHTML is concerned primarily with content rather than the details of how that content is presented by browsers. That is not entirely true, even with the tags discussed in Chapter 2, "Introduction to XHTML." Some of those tags—for example, `<code>`—specify presentation details, or style. However, these presentation specifications can be more precisely and more consistently described with style sheets. Furthermore, many of the tags and attributes used for describing presentation details have been deprecated in favor of style sheets.

Most XHTML tags have associated properties, which store presentation information for browsers. Browsers use default values for these properties if the document does not specify values. For example, the `<h2>` tag has the `font-size` property, for which a browser could have the default value of 18 points. A style sheet could specify that the `font-size` property for `<h2>` be set to 20 points, which would override the default value. The new value could apply to one occurrence of an `<h2>` element or all such occurrences in the document, depending on how the property value is set.

The idea of a style sheet is not new—word processors and desktop publishing systems have long used style sheets to impose a particular style on documents. The first style-sheet specification for use in XHTML documents, dubbed Cascading Style Sheets (CSS1), was developed in 1996 by the W3C. In mid-1998, the second standard, CSS2, was released. CSS2 added many properties and property values to CSS1. It also extended presentation control to media other than Web browsers, such as printers. CSS3 has been under development since the late 1990s. Browser vendors quickly implemented some of the features of CSS1 but have been much slower at getting around to implementing the rest. Most, but not all of CSS1 has been implemented by Internet Explorer 7 (IE7) and FireFox 2 (FX2), but support for CSS2 is far from complete, especially in IE7. As a result of the incomplete implementation (and perhaps a lack of interest) of CSS2, W3C decided to develop a new standard, CSS 2.1, which would reflect the level of acceptance of CSS 2 by the browser implementors. CSS 2.1 is now at the "working draft" stage.

This chapter is restricted to a subset of the CSS1 properties and property values, along with a few from CSS2 that are supported by one or both of the two most popular browsers. Unless otherwise stated, all of the examples in the chapter work correctly for both IE7 and FX2 browsers.

Perhaps the most important benefit of style sheets is their capability of imposing consistency on the style of Web documents. For example, they allow the author to specify that all occurrences of a particular tag use the same presentation style.

XHTML style sheets are called *cascading* style sheets because they can be defined at three different levels to specify the style of a document. Lower-level style sheets can override higher-level style sheets, so the style of the content of a tag is determined in effect through a cascade of style-sheet applications.

## 3.2 Levels of Style Sheets

The three levels of style sheets, in order from lowest level to highest level, are *inline*, *document level*, and *external*. Inline style sheets apply to the content of a single element, document-level style sheets apply to the whole body of a document, and external style sheets can apply to the bodies of any number of documents. Inline style sheets have precedence over document style sheets, which have precedence over external style sheets. For example, if an external style sheet specifies a value for a particular property of a particular tag, that value is used until a different value is specified in either a document style sheet or an inline style sheet. Likewise, document style sheet property values can be overridden by different property values in an inline style sheet. In effect, the properties of a specific tag are those that result from a merge of all applicable style sheets, with lower-level style sheets having precedence in cases of conflicting specifications.There are other ways style specification conflicts can occur. These and their resolution are discussed in Section 3.13.

If no style sheet information is specified, the browser default property values are used.

As is the case with tags and tag attributes, a particular browser may not be capable of using the property values specified in a style sheet. For example, if the value of the `font-size` property of a paragraph is set to 18 points, but the browser can only display the particular font being used in sizes up to 16 points, the browser obviously cannot fulfill the property specification. In this case, the browser either would substitute an alternative value or would simply ignore the given font size value.

Inline style specifications appear within the opening tag and apply only to the content of that tag. This fine-grain application of style defeats one of the primary advantages of style sheets—that of imposing a uniform style on the tags of at least one whole document. Another disadvantage of inline style sheets is that they result in style information, which is expressed in a language distinct from XHTML markup, being embedded in various places in documents. It is much better to keep style specifications separate from XHTML markup. For this reason, among others, W3C deprecated inline style sheets in XHTML 1.1.[1]

1. Being placed on the list of deprecated features is a warning to users to restrict their use, because sometime in the future they will be discontinued.

Therefore, inline style specifications should be used sparingly. This chapter discusses inline style sheets, but we follow our own advice and make little use of them in our examples.

Document-level style specifications appear in the document head section and apply to the entire body of the document. This is obviously the way to impose a uniform style on the presentation of all of the content of a document.

In many cases, it is desirable to have a style sheet apply to more than one document. This is the purpose of external style sheets. External style sheets are not part of any of the documents to which they apply. They are stored separately and are referenced in all documents that use them. External style sheets are written as text files with the MIME type `text/css`. They can be stored on any computer on the Web. The browser fetches external style sheets just as it fetches documents. The `<link>` tag is used to specify external style sheets. Within `<link>`, the `rel` attribute is used to specify the relationship of the linked-to document to the document in which the link appears. The `href` attribute of `<link>` is used to specify the URL of the style sheet document, as in the following example:

```
<link rel = "stylesheet"  type = "text/css"
  href = "http://www.cs.usc.edu/styles/wbook.css" />
```

The link to an external style sheet must appear in the head of the document. If the external style sheet resides on the Web server computer, only its path address must be given as the value of `href`. An example of an external style sheet appears in Section 3.6.

The `@import` directive is an alternative way to use style specifications from other files. The form is the following:

`@import url(`*filename*`);`

Notice that the file name is not quoted. Two differences between `link` and `@import` are: (1) `@import` can appear only at the beginning of the content of a style element, and (2) the imported file can contain markup, as well as style rules. In fact, sometimes the imported file contains other `@import` directives, along with some style rules.

External style sheets can be validated with the service provided at `http://jigsaw.w3.org/css-validator/validator-upload.html`.

## 3.3 Style Specification Formats

The format of a style specification depends on the level of style sheet. Inline style specifications appear as values of the `style` attribute of a tag,[2] the general form of which is as follows:

`style = "`*property_1*`:`*value_1*`; `*property_2*`:`*value_2*`; ...;`
*property_n*`:`*value_n*`;"`

2. The `style` attribute is deprecated in the XHTML 1.1 recommendation.

Although it is not required, it is recommended that the last property/value pair be followed by a semicolon.

Document style specifications appear as the content of a style element within the header of a document, although the format of the specification is quite different from that of inline style sheets. The general form of the content of a style element is as follows:[3]

```
<style type = "text/css">
  rule_list
</style>
```

The `type` attribute of the `<style>` tag tells the browser the type of style specification, which is always `text/css`. The type of style specification is necessary because there are other kinds of style sheets. For example, JavaScript also provides style sheets that can appear in style elements.

Each style rule in a rule list has two parts: a selector, which indicates the tag or tags affected by the rule, and a list of property/value pairs. The list has the same form as the quoted list for inline style sheets, except the list is delimited by braces rather than double quotes. So, the form of a style rule is as follows:

```
selector {property_1:value_1; property_2:value_2; ...;
          property_n:value_n;}
```

If a property is given more than one value, those values usually are separated by spaces. For some properties, however, multiple values are separated with commas.

Like all other kinds of coding, complicated CSS rule lists should be documented with comments. Of course, XHTML comments cannot be used here, because CSS is not XHTML. Therefore, a different form of comment is needed. CSS comments are introduced with `/*` and terminated with `*/`.[4] For example:

```
<style type = "text/css">
  /* Styles for the initial paragraph */
  ...
  /* Styles for other paragraphs */
  ...
</style>
```

External style sheets have a form similar to that of document style sheets. The external file consists of a list of style rules. An example of an external style sheet appears in Section 3.6.

---

3. Browsers so old that they do not recognize the `<style>` tag may display the content of the style element at the top of the document. There are now so few such browsers in use that we ignore the issue here. Those who are concerned put the rule list in an XTHML comment.

4. This form of comment is adopted from the C programming language and some of its descendants.

## 3.4 Selector Forms

The selector can have a variety of forms, which are described in this section.

### 3.4.1 Simple Selector Forms

The simplest selector form is a single element name, such as `h1`. In this case, the property values in the rule apply to all occurrences of the named element. The selector could be a list of element names, separated by commas, in which case the property values apply to all occurrences of all of the named elements. Consider the following examples, in which the property is `font-size` and the property value is a number of points:

```
h1 {font-size: 24pt;}
h2, h3 {font-size: 20pt;}
```

The first of these specifies that the text content of all `h1` elements must be set in 24-point font size. The second specifies that the text content of all `h2` and `h3` elements must be set in 20-point font size.

Selectors can also specify that the style should only apply to elements in certain positions in the document. This is done by listing the element hierarchy in the selector, with only whitespace separating the element names. For example, the rule

```
body b em {font-size: 14pt;}
```

only applies its style to the content of emphasis elements that are descendants of bold elements in the body of the document. This is a *contextual* selector (sometimes called a *descendant* selector). It would not apply to the content of an emphasis element that did not have a boldface element as an ancestor.

### 3.4.2 Class Selectors

Class selectors are used to allow different occurrences of the same tag to use different style specifications. A style class is defined in a style element by giving it a name, which is attached to the tag's name with a period. For example, if you want two paragraph styles in a document—say, `normal` and `warning`—you could define these two classes in the content of a `<style>` tag as follows:

```
p.normal {property-value list}
p.warning {property-value list}
```

Within the document body, the particular style class that you want is specified with the `class` attribute of the affected tag—in the preceding example, the paragraph tag. For example, you might have the following:

```
<p class = "normal">
A paragraph of text that we want to be presented in
'normal' presentation style
```

```
</p>
<p class = "warning">
A paragraph of text that is a warning to the reader, which
should be presented in an especially noticeable
presentation style
</p>
```

### 3.4.3 Generic Selectors

Sometimes it is convenient to have a class of style specifications that applies to the content of more than one kind of tag. This is done by using a generic class, which is defined without a tag name in its name. In place of the tag name, you use the name of the generic class, which must begin with a period. For example:

`.sale` {*property-value list*}

Now, in the body of a document, you could have the following:

```
<h3 class = "sale"> Weekend Sale </h3>
...
<p class = "sale">
...
</p>
```

### 3.4.4 id Selectors

An `id` selector allows the application of a style to one specific element. The general form of an `id` selector is as follows:[5]

#*specific-id* {*property-value list*}

As you can probably guess, the style specified in the `id` selector applies to the element with the specific `id`. For example:

```
#section14 {font-size: 20}
```

specifies a font size of 20 points to the element

```
<h2 id = "section14">1.4 Calico Cats </h2>
```

CSS2 added still more selector forms. However, because of the lack of browser support for them, they are not discussed here.

### 3.4.5 Universal Selectors

The universal selector, denoted by an asterisk (*), applies its style to all elements in the document. For example:

```
* {color: red;}
```

5. For the oddly curious reader, the Bell Labs name for the # symbol is *octothorpe*. It was named that when it was first put on the telephone dial, for some unknown reason.

makes all elements in the document red.

The universal selection is not often useful.

### 3.4.6 Pseudo Classes

Pseudo classes are styles that apply when something happens rather than because the target element simply exists. CSS1 included some pseudo classes, and CSS2 added more. Unfortunately, support for the pseudo classes is sorely lacking, at least among the most popular browsers. However, two pseudo classes, `hover` and `focus`, are supported by FX2, so we introduce them here. IE7 supports `hover`, but not `focus`.

While the names of style classes and generic classes begin with a period, the names of pseudo classes begin with colons. The style of the `hover` pseudo class applies when its associated element has the mouse cursor over it. The style of the `focus` pseudo class applies when its associated element has focus.[6] For example, consider the following document:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- pseudo.html
     Illustrates the :hover and :focus pseudo classes
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Pseudo Classes </title>
    <style type = "text/css">
      input:hover {color: red;}
      input:focus {color: green;}
    </style>
  </head>
  <body>
    <form action = "">
      <p>
        <label>
          Your name:
          <input type = "text"  />
        </label>
      </p>
    </form>
  </body>
</html>
```

6. An element acquires focus when the user places the mouse cursor over it and clicks the left mouse button.

In `pseudo.html`, the content of an input element (a text box) is colored red when the mouse cursor is placed over its content. This happens only when the text box does not have focus. If no text has been typed into the text box, the hover pseudo class has no effect. When the text box acquires focus, the text turns green and stays that color until the left mouse button is clicked outside the box.

## 3.5 Property Value Forms

CSS1 includes 60 different properties in seven categories: fonts, lists, alignment of text, margins, colors, backgrounds, and borders. As you probably will guess, we will not discuss all of these properties. The complete details of all properties and property values can be found at the W3C Web site.

Property values can appear in a variety of forms. Keyword property values are used when there are only a few possible values and they are predefined—for example, `large`, `medium`, and `small`. Keyword values are not case sensitive, so `Small`, `SmAlL`, and `SMALL` are all the same as `small`.

Number values are used when no meaningful units can be attached to a numeric property value. A number value either can be an integer or a sequence of digits with a decimal point and can be preceded by a sign (+ or -).

Length values are specified as number values that are followed immediately by a two-character abbreviation of a unit name. There can be no space between the number and the unit name. The possible unit names are `px` for pixels, `in` for inches, `cm` for centimeters, `mm` for millimeters, `pt` for points (a point is 1/72 inch), and `pc` for picas, which are 12 points. Note that on a display, the `in`, `cm`, `mm`, `pt`, and `pc` are approximate measures. Their actual values depend on screen resolution. There are also two relative length values: `em`, which is the value of the current font size in pixels, and `ex`, which is the height of the letter *x*.

Percentage values are used to provide a measure that is relative to the previously used measure for a property value. Percentage values are numbers that are followed immediately by a percent sign. For example, if the font size were set to `75%`, it would make the new current size for the font 75 percent of its previous value. Font size would stay at the new value until changed again. Percentage values can be signed. If preceded by a plus sign, the percentage is added to the previous value; if negative, the percentage is subtracted.

URL property values use a form that is slightly different from references to URLs in links. The actual URL, which can be either absolute or relative, is placed in parentheses and preceded by `url`, as in the following:

```
url(tetons.jpg)
```

There can be no space between `url` and the left parenthesis.

Color property values can be specified as color names, as six-digit hexadecimal numbers, or in RGB form. RGB form is just the word `rgb` followed by a parenthesized list of three numbers that specify the levels of red, green, and blue. The RGB values can be given as either decimal numbers between 0 and 255 or as percentages. Hexadecimal numbers must be preceded with pound signs (#), as in `#43AF00`. For example, powder blue could be specified with

```
powderblue
```

or

```
rgb(176, 224, 230)
```

or

```
#B0E0E6
```

CSS2 specifies that some property values are inherited by elements nested in the element for which the values are specified. For example, the property `background-color` is not inherited but `font-size` is. Using a style sheet to set a value for an inheriting property for the `<body>` tag effectively sets it as a default property value for the whole document. For example:

```
body {font-size: 16pt}
```

Unless overridden by a style sheet that applies to paragraph elements, every paragraph element in the body of this document would inherit the font size of 16 points.

## 3.6 Font Properties

The font properties are among the most commonly used of the style-sheet properties. Virtually all XHTML documents include text, which is often used in a variety of different situations. This creates a need for text in many different fonts, font styles, and sizes. The font properties allow us to specify these different forms.

### 3.6.1 Font Families

The `font-family` property is used to specify a list of font names. The browser will use the first font in the list that it supports. For example, the following could be specified:

```
font-family: Arial, Helvetica, Futura
```

In this case, the browser will use Arial if it supports that font. If not, it will use Helvetica if it supports it. If the browser supports neither Arial nor Helvetica, it will use Futura if it can. If the browser does not support any of the specified fonts, it will use an alternative of its choosing.

A generic font can be specified as a `font-family` value. The possible generic fonts and examples of each are shown in Table 3.1. Each browser has a font defined for each of these generic names. A good approach to specifying fonts is to use a generic font as the last font in the value of a `font-family` property. For example, because Arial, Helvetica, and Futura are sans-serif fonts,[7] the example above would be better as follows:

```
font-family: Arial, Helvetica, Futura, sans-serif
```

7. Serifs are non-structural decorations that may appear at the ends of strokes in a character. Sans-serif fonts do not have serifs.

**Table 3.1** Generic fonts

| Generic Name | Examples |
|---|---|
| `serif` | `Times New Roman, Garamond` |
| `sans-serif` | `MS Arial, Helvetica` |
| `cursive` | `Caflisch Script, Zapf-Chancery` |
| `fantasy` | `Critter, Cottonwood` |
| `monospace` | `Courier, Prestige` |

If a font name has more than one word, the whole name should be delimited by single quotes,[8] as in the following example:

```
font-family: 'Times New Roman'
```

In practice, the quotes may not be mandatory, but their use is recommended because they may be required in the future.

### 3.6.2 Font Sizes

The `font-size` property does what its name implies. For example, the following property specification sets the font size for text to 10 points:

```
font-size: 10pt
```

Many relative `font-size` values are defined; namely, `xx-small`, `x-small`, `small`, `medium`, `large`, `x-large`, and `xx-large`. In addition, `smaller` or `larger` can be specified. Furthermore, the value can be a percentage, which would be relative to the current font size.

The disadvantage of the relative font sizes is the lack of strict font size control. Different browsers can use different values for them. For example, `small` might mean 10 points on one browser and 8 points on another. On the other hand, using a specific font size has the risk that some browsers may not support the particular size, causing the document to appear different on different browsers.

### 3.6.3 Font Variants

The default value of the `font-variant` property is `normal`, which specifies the usual character font. This property can be set to `small-caps` to specify small capital characters. Small cap characters are all uppercase, but the letters that are normally uppercase are somewhat larger than those that are normally lowercase.

8. We use single quotes here because, in the case of inline style sheets, the whole property list is delimited by double quotes.

### 3.6.4 Font Styles

The `font-style` property is most commonly used to specify italic, as in the following:

```
font-style: italic
```

An alternative to `italic` is `oblique`, but when displayed, the two are nearly identical, so `oblique` is not a terribly useful font style. In fact, some browsers do not support oblique, so they display all oblique fonts in italic.

### 3.6.5 Font Weights

The `font-weight` property is used to specify the degree of boldness. For example:

```
font-weight: bold
```

Besides `bold`, the values `normal` (the default), `bolder`, and `lighter` can be specified. The `bolder` and `lighter` values are taken as relative to the current level of boldness. Specific numbers also can be given in multiples of 100 from 100 to 900, where `400` is the same as `normal` and `700` is the same as `bold`.

### 3.6.6 Font Shorthands

If more than one font property must be specified, the values can be stated in a list as the value of the `font` property—the browser has the responsibility for determining from the forms of the values which properties to assign. For example, consider the following specification:

```
font: bold 14pt 'Times New Roman' Palatino
```

This specifies that the font weight should be `bold`, the font size should be 14 points, and either Times New Roman or Palatino font should be used, with precedence given to Times New Roman.

The order in which the property values are given in a `font` value list is important. The order must be as follows: the font names must be last, the font size must be second last, and the font style, font variant, and font weight, when they are included, can be in any order but must precede the font size and font names. Only the font size and the font family are required in the `font` value list.

The sample XHTML document opposite illustrates some aspects of style-sheet specification of the font properties in headings and paragraphs using a document style sheet.

Figure 3.1 shows a browser display of `fonts.html`.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- fonts.html
     An example to illustrate font properties
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Font properties </title>
    <style type = "text/css">
      p.major {font-size: 14pt;
               font-style: italic;
               font-family: 'Times New Roman';
              }
      p.minor {font: 10pt bold 'Courier New';}
      h2 {font-family: 'Times New Roman';
          font-size: 24pt; font-weight: bold}
      h3 {font-family: 'Courier New'; font-size: 18pt}
    </style>
  </head>
  <body>
    <p class = "major">
      If a job is worth doing, it's worth doing right.
    </p>
    <p class = "minor">
      Two wrongs don't make a right, but they certainly
      can get you in a lot of trouble.
    </p>
    <h2> Chapter 1 Introduction </h2>
    <h3> 1.1 The Basics of Computer Networks </h3>
  </body>
</html>
```

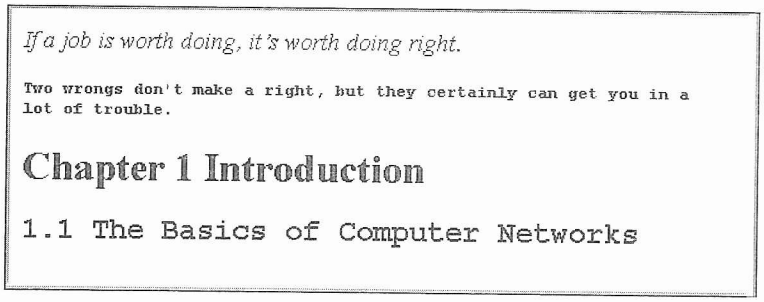


**Figure 3.1** Display of `fonts.html`

The following is a revision of `fonts.html`, `fonts2.html`, which uses an external style sheet in place of the document style sheet used in `fonts.html`. The external style sheet, `styles.css`, follows the revised document.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- fonts2.html
     An example to test external style sheets
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> External style sheets </title>
    <link rel = "stylesheet"  type = "text/css"
          href = "styles.css" />
  </head>
  <body>
    <p class = "major">
      If a job is worth doing, it's worth doing right.
    </p>
    <p class = "minor">
      Two wrongs don't make a right, but they certainly
      can get you in a lot of trouble.
    </p>
    <h2> Chapter 1 Introduction </h2>
    <h3> 1.1 The Basics of Computer Networks </h3>
  </body>
</html>
```

```
/* styles.css - an external style sheet
     for use with fonts2.html
   */
  p.major {font-size: 14pt;
           font-style: italic;
           font-family: 'Times New Roman';
         }
  p.minor {font: 10pt bold 'Courier New';}
  h2 {font-family: 'Times New Roman';
      font-size: 24pt; font-weight: bold}
  h3 {font-family: 'Courier New';
      font-size: 18pt}
```

### 3.6.7 Text Decoration

The `text-decoration` property is used to specify some special features of text. The available values are `line-through`, `overline`, `underline`, and `none`, which is the default. Many browsers underline links. The `none` value can be used to avoid this. Note that `text-decoration` is not inherited. The following example illustrates the `line-through`, `overline`, and `underline` values.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- decoration.html
     An example that illustrates several of the
     possible text decoration values
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Text decoration </title>
    <style type = "text/css">
      p.delete {text-decoration: line-through}
      p.cap {text-decoration: overline}
      p.attention {text-decoration: underline}
    </style>
  </head>
  <body>
    <p class = "delete">
      This illustrates line-through
    </p>
    <p class= "cap">
      This illustrates overline
    </p>
    <p class = "attention">
      This illustrates underline
    </p>
  </body>
</html>
```

Figure 3.2 shows a browser display of `decoration.html`.

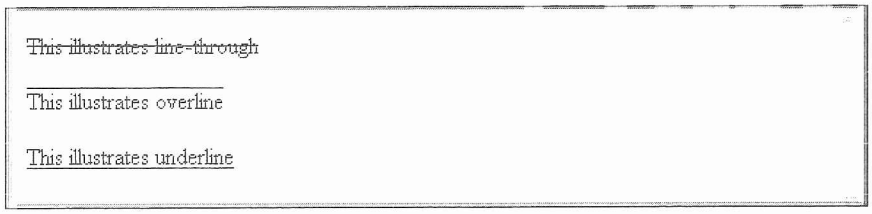


**Figure 3.2** Display of `decoration.html`

The `letter-spacing` property controls the amount of space between characters in text. The possible values of `letter-spacing` are any length property values, for example `3px`.

## 3.7 List Properties

Often, two presentation details of lists are specified in XHTML documents: the shape of the bullets that precede the items in an unordered list and the sequencing values that precede the items in ordered lists. The `list-style-type` property is used to specify both of these.

The `list-style-type` property of an unordered list can be set to `disc`, `circle`, `square`, or `none`. The default property value for bullets is `disc`. For example, the following illustrates a document style sheet to set the bullet type in all items in unordered lists to `square`:

```
<!-- bullets1 -->
<style type = "text/css">
  ul {list-style-type: square}
</style>
...
<h3> Some Common Single-Engine Aircraft </h3>
  <ul>
    <li> Cessna Skyhawk </li>
    <li> Beechcraft Bonanza </li>
    <li> Piper Cherokee </li>
  </ul>
```

Style classes can be defined to allow different list items to have different bullet types:

```
<!-- bullets2 -->
<style type = "text/css">
  li.disc {list-style-type: disc}
  li.square {list-style-type: square}
```

```
  li.circle {list-style-type: circle}
</style>
...
<h3> Some Common Single-Engine Aircraft </h3>
  <ul>
    <li class = "disc"> Cessna Skyhawk </li>
    <li class = "square"> Beechcraft Bonanza </li>
    <li class = "circle"> Piper Cherokee </li>
  </ul>
```

Figure 3.3 shows a browser display of these two lists.

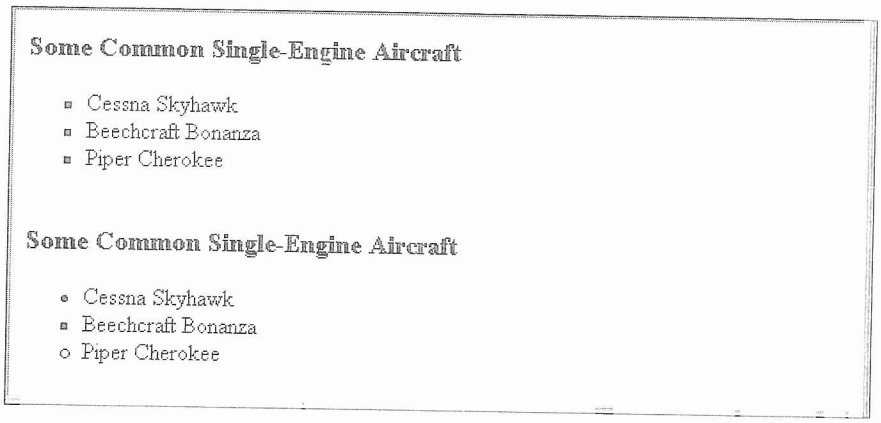


**Figure 3.3** Examples of unordered lists

We are not limited to discs, squares, and circles for bullets in unordered lists. Any image can be used in a list item bullet. Such a bullet is specified with the `list-style-image` property, whose value is specified with the `url` form. For example, if `small_plane.gif` is a small image of an airplane that is stored in the same directory as the XHTML document, it could be used as follows:

```
<style type = "text/css">
  li.image {list-style-image: url(small_airplane.gif)}
</style>
  ...
  <li class = "image"> Beechcraft Bonanza </li>
```

When ordered lists are nested, it is best to use different kinds of sequence values for the different levels of nesting. The `list-style-type` property can be used to specify the types of sequencing values. Table 3.2 lists the different possibilities defined by CSS1.

**Table 3.2** Possible sequencing values for ordered lists

| Property Values | Sequence Type | First Four Values |
|---|---|---|
| `decimal` | Arabic numerals | 1, 2, 3, 4 |
| `upper-alpha` | Uppercase letters | A, B, C, D |
| `lower-alpha` | Lowercase letters | a, b, c, d |
| `upper-roman` | Uppercase Roman numerals | I, II, III, IV |
| `lower-roman` | Lowercase Roman numerals | i, ii, iii, iv |

The following example illustrates the use of different sequence value types in nested lists.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- sequence_types.html
     An example to illustrate sequence type styles
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Sequence types </title>
    <style type = "text/css">
      ol {list-style-type: upper-roman;}
      ol ol {list-style-type: upper-alpha;}
      ol ol ol {list-style-type: decimal;}
    </style>
  </head>
  <body>
    <h3> Aircraft Types </h3>
    <ol>
      <li> General Aviation (piston-driven engines)
        <ol>
          <li> Single-Engine Aircraft
            <ol>
              <li> Tail wheel </li>
              <li> Tricycle </li>
            </ol>
          </li>
          <li> Dual-Engine Aircraft
            <ol>
              <li> Wing-mounted engines </li>
```

```
            <li> Push-pull fuselage-mounted engines </li>
          </ol>
        </li>
      </ol>
    </li>
    <li> Commercial Aviation (jet engines)
      <ol>
        <li> Dual-Engine
          <ol>
            <li> Wing-mounted engines </li>
            <li> Fuselage-mounted engines </li>
          </ol>
        </li>
        <li> Tri-Engine
          <ol>
            <li> Third engine in vertical stabilizer </li>
            <li> Third engine in fuselage </li>
          </ol>
        </li>
      </ol>
    </li>
  </ol>
 </body>
</html>
```

Figure 3.4 shows a browser display of `sequence_types.html`.

**Aircraft Types**

I. General Aviation (piston-driven engines)
   A. Single-Engine Aircraft
      1. Tail wheel
      2. Tricycle
   B. Dual-Engine Aircraft
      1. Wing-mounted engines
      2. Push-pull fuselage-mounted engines

II. Commercial Aviation (jet engines)
   A. Dual-Engine
      1. Wing-mounted engines
      2. Fuselage-mounted engines
   B. Tri-Engine
      1. Third engine in vertical stabilizer
      2. Third engine in fuselage

**Figure 3.4** Display of `sequence_types.html`

CSS2 added more sequence types such as `hebrew` and `lower-greek`, but they are not yet supported by the popular browsers.

## 3.8 Color

If older browsers and older client machines are taken into account, color is not a simple issue. For one thing, the document may be displayed on monitors of widely varying capabilities. Also, the document may be rendered by browsers that have different abilities to deal with colors. This section provides an introduction to how Web sites can deal with these difficulties.

### 3.8.1 Color Groups

Three levels of collections of colors might be used by an XHTML document. The smallest useful set of colors includes only those that have standard names and are guaranteed to be correctly displayable by all browsers on all color monitors. This collection of sixteen colors is called the *named colors*. The names and hexadecimal codes for the named colors are shown in Table 3.3.

**Table 3.3** Named colors

| Name | Hexadecimal Code | Name | Hexadecimal Code |
|---|---|---|---|
| black | 000000 | green | 008000 |
| silver | C0C0C0 | lime | 00FF00 |
| gray | 808080 | olive | 808000 |
| white | FFFFFF | yellow | FFFF00 |
| maroon | 800000 | navy | 000080 |
| red | FF0000 | blue | 0000FF |
| purple | 800080 | teal | 008080 |
| fuchsia | FF00FF | aqua | 00FFFF |

Most Web browsers now recognize 140 named colors, although these names are not part of a W3C standard. This collection of colors is given in Appendix B.

A larger set of colors, called the *Web palette*, includes 216 colors. These colors, which are often called Web-safe colors, are displayable by Windows- and Macintosh-based browsers but may not be correctly displayed with some older terminals used on UNIX systems. Elements of this set of colors have hexadecimal values for red, green, and blue that are restricted to `00`, `33`, `66`, `99`, `CC`, and `FF`. These numbers allow all combinations of all increments of 20 percent of

each of the three basic colors, red, green, and blue. The colors of the Web palette are shown on the back inside cover of this book.

When the limitations of older browsers and monitors are not a consideration, 24-bit (or six-hexadecimal-digit) numbers can be used to specify any one of 16 million colors. When a color is specified that the browser or monitor cannot display, a (hopefully) similar color will be used.

### 3.8.2 Color Properties

The `color` property is used to specify the foreground color of XHTML elements. For example, consider the following small table:

```
<style type = "text/css">
  th.red {color: red}
  th.orange {color: orange}
</style>
  ...
<table border = "5px">
  <tr>
    <th class = "red"> Apple </th>
    <th class = "orange"> Orange </th>
    <th class = "orange"> Screwdriver </th>
  </tr>
</table>
```

The `background-color` property is used to set the background color of an element, where the element could be the whole body of the document. For example, consider the following paragraph element:

```
<style type = "text/css">
  p.standout {font-size: 24pt; color: blue;
              background-color: red">
</style>
...
<p class = "standout">
  To really make it stand out, use a red background!
</p>
```

When displayed by a browser, this might appear as shown in Figure 3.5.

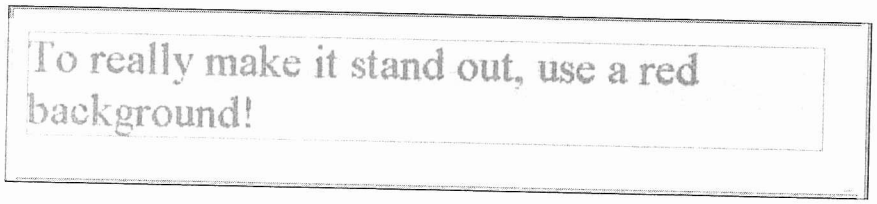


**Figure 3.5** The `background-color` property

## 3.9 Alignment of Text

The first line of a paragraph can be indented using the `text-indent` property. This property takes either a length or a percentage value. For example:

```
<style type = "text/css">
  p.indent {text-indent: 0.5in}
</style>
...
<p class = "indent">
  Now is the time for all good Web programmers to begin
  using cascading style sheets for all presentation
  details in their documents. No more deprecated tags
  and attributes, just nice, precise style sheets.
</p>
```

This paragraph would be displayed as follows:

> Now is the time for all good Web programmers to begin using cascading style sheets for all presentation details in their documents. No more deprecated tags and attributes, just nice, precise style sheets.

The `text-align` property, for which the possible keyword values are `left`, `center`, `right`, and `justify`, is used to arrange text horizontally. For example, the following document-level style sheet entry causes the content of paragraphs to be aligned on the right margin:

```
p {text-align: right}
```

The default value for `text-align` is `left`.

The `float` property, which is often set for images and tables, is used to specify that text should flow around some element. The possible values for `float` are `left`, `right`, and `none`, which is the default. For example, suppose we want an image to be on the right side of the display and have text flow around its left side. To specify this, the `float` property of the image is set to `right`. Because the default value for `text-align` is `left`, `text-align` need not be set for the text. In the following example, the text of a paragraph is specified to flow to the left of an image until the bottom of the image is reached, at which point the paragraph text flows across the whole window.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- float.html
     An example to illustrate the float property
     -->
```

```
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> The float property </title>
    <style type = "text/css">
      img {float: right}
    </style>
  </head>
  <body>
    <p>
      <img src = "c210new.jpg"  alt = "Picture of a Cessna 210" />
    </p>
    <p>
      This is a picture of a Cessna 210. The 210 is the flagship
      single-engine Cessna aircraft. Although the 210 began as a
      four-place aircraft, it soon acquired a third row of seats,
      stretching it to a six-place plane. The 210 is classified
      as a high-performance airplane, which means its landing
      gear is retractable and its engine has more than 200
      horsepower. In its first model year, which was 1960,
      the 210 was powered by a 260-horsepower fuel-injected
      six-cylinder engine that displaced 471 cubic inches.
      The 210 is the fastest single-engine airplane ever
      built by Cessna.
    </p>
  </body>
</html>
```

When rendered by a browser, `float.html` might appear as shown in Figure 3.6, depending on the width of the browser display window.

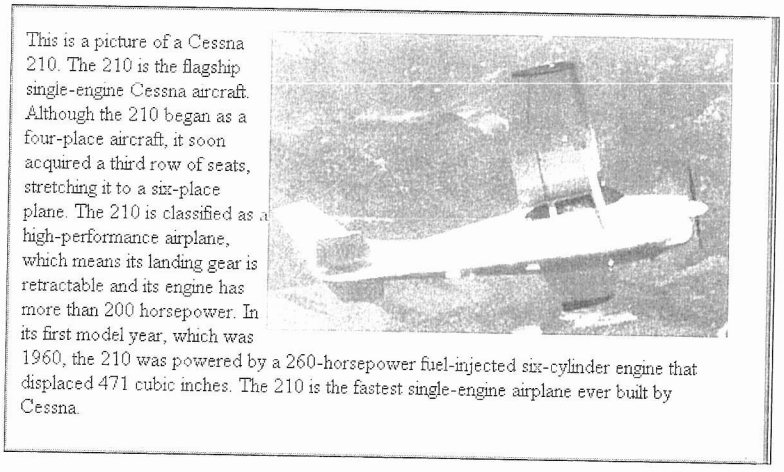


**Figure 3.6** Display of `float.html`

## 3.10 The Box Model

Virtually all document elements can have borders. These borders have various styles such as color and width. Furthermore, the amount of space between the content of an element and its border, known as *padding*, can be specified, as well as the space between the border and an adjacent element, known as the *margin*. This model is shown in Figure 3.7.

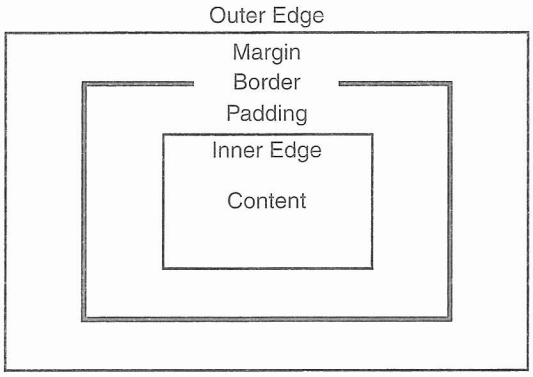


**Figure 3.7** The box model

### 3.10.1 Borders

Every element has a property, `border-style`, that controls whether the element's content has a border, as well as the style of the border.[9] CSS1 requires that borders be available for any element, but the only required style is `solid` (the default style when the border attribute of a table element is set to `border` or a pixel width). CSS2 provides several different border styles, among them `dotted`, `dashed`, and `double`, all of which are supported by IE7 and FX2. The default value for `border-style` is `none`, which is why the contents of elements do not normally have borders. The styles of one of the four sides of an element can be set with `border-top-style`, `border-bottom-style`, `border-left-style`, and `border-right-style`.

The `border-width` property is used to specify the thickness of a border. Its possible values are `thin`, `medium` (the default), `thick`, or a length value in pixels. Setting `border-width` sets the thickness of all four sides of an element. The width of each of the four borders of an element can be different. These are specified with `border-top-width`, `border-bottom-width`, `border-left-width`, and `border-right-width`. All of the border width properties are part of CSS1.

9. Recall that the border on a table can be set with the `border` attribute of the table element.

The color of a border is controlled by the `border-color` property, which is part of CSS1. Once again, the individual borders of an element can be colored differently through the CSS2 properties, `border-top-color`, `border-bottom-color`, `border-left-color`, and `border-right-color`. All of the border color properties are supported by IE7 and FX2.

The following document, `borders.html`, illustrates borders, using a table and a short paragraph as examples. Notice that if a table has a border that was

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- borders.html
     An example of a simple table with various borders
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Table borders </title>
    <style type = "text/css">
      table {border-top-width: medium;
             border-bottom-width: thick;
             border-top-color: red;
             border-bottom-color: blue;
             border-top-style: dotted;
             border-bottom-style: dashed;
            }
      p {border-style: dashed; border-width: thin;
         border-color: green
        }
    </style>
  </head>
  <body>
    <table border = "5">
      <caption> Fruit Juice Drinks </caption>
      <tr>
        <th> </th>
        <th> Apple </th>
        <th> Orange </th>
        <th> Screwdriver </th>
      </tr>
      <tr>
        <th> Breakfast </th>
        <td> 0 </td>
        <td> 1 </td>
        <td> 0 </td>
      </tr>
```

```
      <tr>
        <th> Lunch </th>
        <td> 1 </td>
        <td> 0 </td>
        <td> 0 </td>
      </tr>
      <tr>
        <th> Dinner </th>
        <td> 0 </td>
        <td> 0 </td>
        <td> 1 </td>
      </tr>
    </table>
    <p>
      Now is the time for all good Web programmers to
      learn to use style sheets.
    </p>
  </body>
</html>
```

specified with its `border` attribute, the border properties override the original border. In this example, the table has a regular 5 pixel border, but the top and bottom borders are replaced by those specified with the border properties.

The display of `borders.html` is shown in Figure 3.8.

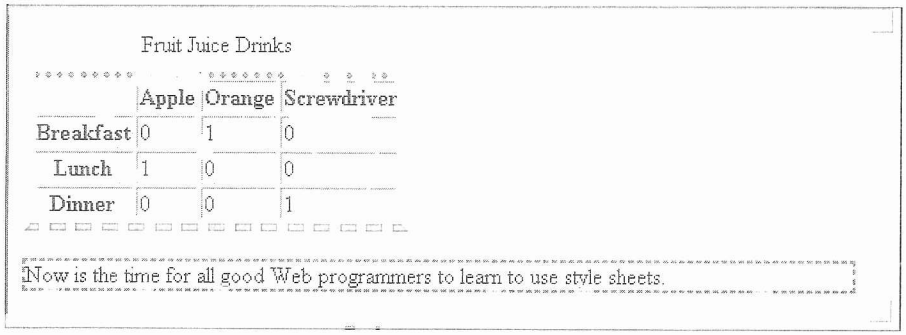


**Figure 3.8** Borders

If the border attribute had been left out of the table element in `borders.html`, the table would have a top border and a bottom border only. It would not have left and right borders, nor would it have borders around the cells.

### 3.10.2 Margins and Padding

Recall from the box model that *padding* is the space between the content of an element and its border. The *margin* is the space between the border of an element and the element's neighbor. When there is no border, the margin plus the padding is the space between the content of an element and its neighbor. In this scenario, it may appear that there is no difference between padding and margins. However, there is a difference when the element has a background. In this case, the background extends into the padding but not into the margin.

The margin properties are named `margin`, which applies to all four sides of an element, `margin-left`, `margin-right`, `margin-top`, and `margin-bottom`. The padding properties are named `padding`, which applies to all four sides, `padding-left`, `padding-right`, `padding-top`, and `padding-bottom`.

The following example, `marpads.html`, illustrates several combinations of margins and padding, both with and without borders.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- marpads.html
     An example to illustrate margins and padding
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Margins and Padding </title>
    <style type = "text/css">
      p.one   {margin: 0.2in;
               padding: 0.2in;
               background-color: #C0C0C0;
               border-style: solid;
              }
      p.two   {margin: 0.1in;
               padding: 0.3in;
               background-color: #C0C0C0;
               border-style: solid;
              }
      p.three {margin: 0.3in;
               padding: 0.1in;
               background-color: #C0C0C0;
               border-style: solid;
              }
```

```
      p.four  {margin:0.4in;
               background-color: #C0C0C0;}
      p.five  {padding: 0.4in;
               background-color: #C0C0C0;
              }
    </style>
  </head>
  <body>
    <p>
      Here is the first line.
    </p>
    <p class = "one">
      Now is the time for all good Web programmers to
      learn to use style sheets. <br /> [margin = 0.2in,
      padding = 0.2in]
    </p>
    <p class = "two">
      Now is the time for all good Web programmers to
      learn to use style sheets. <br /> [margin = 0.1in,
      padding = 0.3in]
    </p>
    <p class = "three">
      Now is the time for all good Web programmers to
      learn to use style sheets. <br /> [margin = 0.3in,
      padding = 0.1in]
    </p>
    <p class = "four">
      Now is the time for all good Web programmers to
      learn to use style sheets. <br /> [margin = 0.4in,
      no padding, no border]
    </p>
    <p class = "five">
      Now is the time for all good Web programmers to
      learn to use style sheets. <br /> [padding = 0.4in,
      no margin, no border]
    </p>
    <p>
      Here is the last line.
    </p>
  </body>
</html>
```

Figure 3.9 shows a browser display of `marpads.html`.

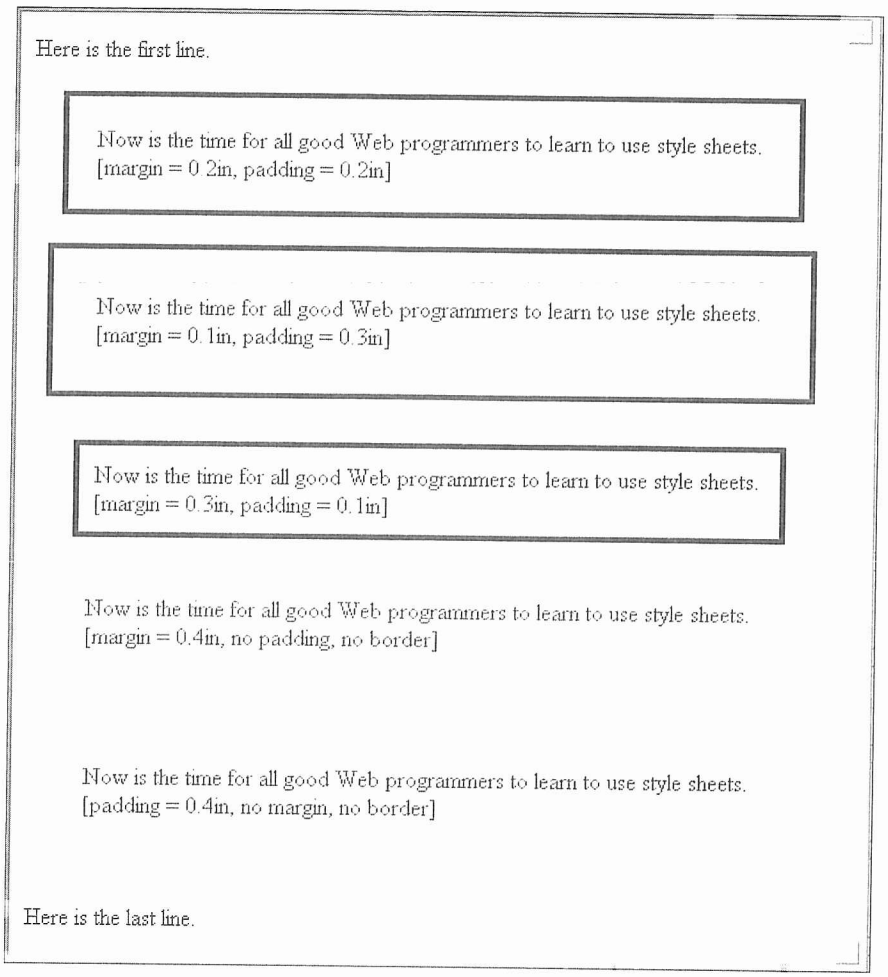


**Figure 3.9** Display of `marpads.html`

## 3.11 Background Images

The `background-image` property is used to place an image in the background of an element. For example, an image of an airplane might be an effective background for text about the airplane. Consider the following example:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
```

```
<!-- back_image.html
     An example to illustrate background images
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Background images </title>
    <style type = "text/css">
      body {background-image: url(c172.gif);}
      p {margin-left: 30px; margin-right: 30px;
         margin-top: 50px; font-size: 14pt;}
    </style>
  </head>
  <body>
    <p>
      The Cessna 172 is the most common general aviation airplane
      in the world. It is an all-metal, single-engine piston,
      high-wing four-place monoplane. It has fixed-gear and is
      categorized as a non-high-performance aircraft. The current
      model is the 172R.
      The wingspan of the 172R is 36'1". Its fuel capacity is 56
      gallons in two tanks, one in each wing. The takeoff weight
      is 2,450 pounds. Its maximum useful load is 837 pounds.
      The maximum speed of the 172R at sea level is 142 mph.
      The plane is powered by a 360 cubic inch gasoline engine
      that develops 160 horsepower. The climb rate of the 172R
      at sea level is 720 feet per minute.
    </p>
  </body>
</html>
```

Figure 3.10 shows a browser display of `back_image.html`.

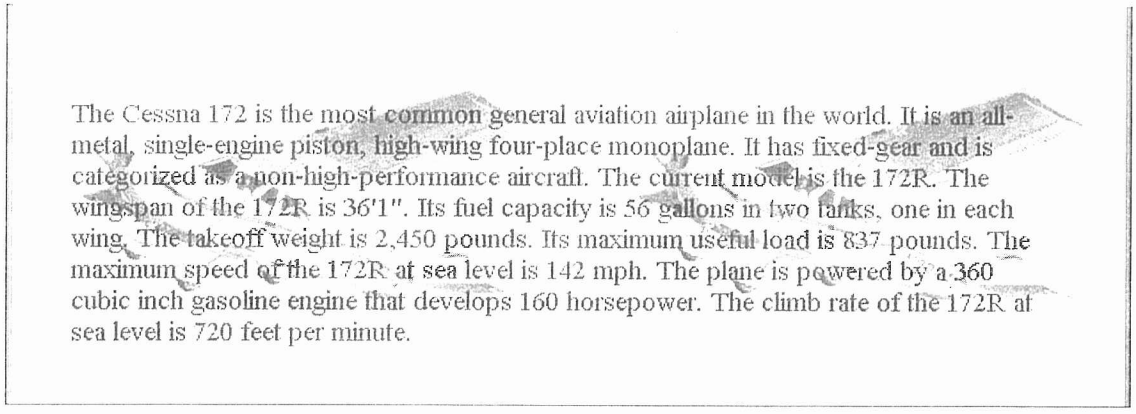


**Figure 3.10** Display of `back_image.html`

Text over a background image can be difficult to read if the image has areas that are nearly the same color as the text. Therefore, care must be taken in selecting background images. In many cases, images of various kinds of textures in light gray colors are best.

In the example, notice that the background image is replicated as necessary to fill the area of the element. This replication is called *tiling*. Tiling can be controlled with the `background-repeat` property, which can take the value `repeat` (the default), `no-repeat`, `repeat-x`, or `repeat-y`. The `no-repeat` value specifies that just one copy of the image is to be displayed. The `repeat-x` value means that the image is to be repeated horizontally; `repeat-y` means to repeat vertically. Additionally, the position of a nonrepeated background image can be specified with the `background-position` property, which can take a large number of different values. The keyword values are `top`, `center`, `bottom`, `left`, and `right`. These can be used in many different combinations. The simplest uses of these are to use one keyword to specify the horizontal placement and one to specify the vertical placement, such as `top left`, `bottom right`, and `top center`. If only one keyword is given, the other is assumed to be `center`. So, `top` is equilivent to `top center` (or `center top`), and `left` is the same as `center left` (or `left center`).

## 3.12 The  and  Tags

In many situations, we want to apply special font properties to less than a whole paragraph of text. For example, it is often useful to have a word or phrase in a line appear in a different font size or color. The `<span>` tag is designed for just this purpose. Unlike most other tags, there is no default layout for the content of `<span>`. So, in the following example, the word `total` is not displayed differently from the rest of the paragraph:

```
<p>
  It sure is fun to be in <span> total </span>
  control of text
</p>
```

The purpose of `<span>` is to change property values of part of a line of content. For example:

```
<style type = "text/css" >
  .spanred {font-size: 24pt;
            font-family: Ariel; color: red}
</style>
...
<p>
  It sure is fun to be in
  <span class = "spanred"> total </span>
  control of text
</p>
```

The display of this paragraph is shown in Figure 3.11.

**Figure 3.11** The `<span>` tag

It is common for documents to have sections, each consisting of some number of paragraphs, that have their own presentation styles. Using style classes on paragraphs, you can do this with what we have already discussed. It is more convenient, however, to be able to apply a style to a section of a document rather than each paragraph. This can be done with the `<div>` tag. As with `<span>`, there is no implied layout for the content of the `<div>` tag, so its primary use is to specify presentation details for a section or division of a document.

Consider the following example, in which a section, or division, of a document is to use a specific paragraph style:

```
<div class = "primary">
  <p>
  ...
  </p>
  <p>
  ...
  </p>
  <p>
  ...
  </p>
</div>
```

The `span` and `div` elements are further shown in examples in Chapter 6.

## 3.13 Conflict Resolution

When there are two different values for the same property on the same element, there is an obvious conflict that the browser (or other XHTML processor) must resolve. So far, we have only considered one way in which this conflict can occur: when style sheets at two or more levels specify different values for the same property on the same element. This particular kind of conflict is resolved by the precedence of the three different levels of style sheets. In-line style sheets have precedence over document and external style sheets, and document style sheets have precedence over external style sheets. However, property value conflicts can occur in several other ways. For example, a conflict may occur within a single style sheet. Consider the following style specifications, which are next to each other in the same document-level style sheet:

```
h3 {color: blue;)
body h3 {color: red;)
```

Both of these apply to all `h3` elements in the body of the document.

Inheritance is another source of potential property value conflicts. These can occur if a property on a particular element has a property value assigned by some style sheet and also inherits a value for that same property. Therefore, some method of resolving conflicts caused by inheritance must be available.

There can be several different origins of the specification of property values. For example, they may come from a style sheet written by the author of the document itself, but they may also come from the browser user and from the browser. For example, an FX2 user can set a minimum font size in the *Tools-Options-Advanced* window. Furthermore, browsers allow their users to write and use their own style sheets. Property values with different origins can have different precedences.

In addition, every property value specification has a particular specificity, depending on the particular kind of selector that is used to set it, and those specificities have different levels of precedence. These different levels are used to resolve conflicts among different specifications.

Finally, property value specifications can be marked as being important, by including `!important` in the specification. For example,

```
p.special {font-style: italic !important; font-size: 14}
```

In this specification, `font-style: italic` is important, but `font-size: 14` is normal. Whether a specification has been marked as being important is called the *weight* of the specification. The weight can be either normal or important. Obviously, this is another way to specify the relative precedence that a specification should have in resolving conflicts.

The details of property value conflict resolution, which are complex, will not be discussed here. Rather, the following is a relatively brief overview of process of property value conflict resolution.

Conflict resolution is a multistage sorting process. The first step in the process is to gather the syle specifications from the three possible levels of style sheets. These specifications are sorted into order by the relative precedence of the style sheet levels. Next, all of the available specifications (those from style sheets, those from the user, and those from the browser) are sorted by origin and weight. This is done according to the following rules, in which the first has the highest precedence

1. Important declarations with user origin
2. Important declarations with author origin
3. Normal declarations with author origin
4. Normal declarations with user origin
5. Any declarations with browser (or other user agent) origin

Note that user-origin specifications are considered to have the highest precedence. The rationale for this is that such specifications often are declared because of some diminished capability of the user; for example, a visual impairment.

If there are conflicts after the sorting described above, the next step in their resolution is a sort by specificity. This sort is based on the following rules, in which the first has the highest precedence:

1. `id` selectors
2. Class and pseudo-class selectors
3. Contextual selectors (more element type names means they are more specific)
4. Universal selectors

If there are still conflicts, they are resolved by giving precedence to the most recently seen specification. For this process, the specifications in an external style sheet are considered to occur at the point in the document where the link element or `@import` rule that references the external style sheet appears. For example, if a style sheet specifies the following, and there are no further conflicting specifications before the element is displayed, the value used will be the last (in this case, `10pt`):

```
p {font-size: 12pt}
p {font-size: 10pt}
```

The whole sorting process that is used to resolve style specification conflicts is called *the cascade*.

## Summary

Cascading style sheets were introduced to provide a uniform and consistent way to specify presentation details in XHTML documents. Many of the style tags and attributes designed for specifying style that had crept into HTML were deprecated in HTML 4.0 in favor of style sheets. Style sheets can appear at three levels: inline, which apply only to the content of one specific tag; document, which apply to all appearances of specific tags in the body of a document; and external, which are stored in files by themselves and can apply to any number of documents. The property values in inline style sheets are specified in the string value of the `style` attribute. Document style sheets are specified in a comment that is the content of a `<style>` tag in the head of the document. External style sheets appear in separate files. Both document-level and external style specifications have the form of a list of style rules. Each style rule has a list of the names of tags and a list of property/value pairs. The property/value pairs apply to all occurrences of the named tags.

A style class, which is defined in the content of a `<style>` tag, allows different occurrences of the same tag to have different property values. A generic style-class specification allows tags with different names to use the same presentation style. A pseudo class takes effect when a particular event occurs. There are many different property value forms, including lengths, percentage values, URLs, and colors. Several different properties are related to fonts. The `font-family` property specifies one or more font names. Because different browsers support different sets of fonts, there are five generic font names. Each browser supports at least one font in each generic category. The `font-size` property can specify a length value or one of a number of different named size categories. The `font-style` property can be set to `italic` or `normal`. The `font-weight` property is used to specify the degree of boldness of text. The `font` property provides an abbreviated form for font-related properties. The `text-decoration` property is used to specify underlining, overlining, and line-through text.

The `list-style-type` property is used to specify the bullet form for items in unordered lists. It is also used to specify the sequence type for the items in ordered lists.

A Web content designer must be concerned with the color capabilities of clients' browsers and monitors. The safest set of colors includes just 16 basic colors. A much larger set of relatively safe Web colors is the Web palette, which includes 216 colors. The foreground and background colors of the content of a document are specified by the `color` and `background-color` properties, respectively.

The first line of a paragraph can be indented with `text-indent`. Text can be aligned with the `text-align` property, whose values are `left`, `right`, and `justify`, which means both left and right alignment. When the `float` property is set to `left` or `right`, text can be made to flow around it on the right or left, respectively, in the display window.

Borders can be specified to appear around any element. These borders can appear in any color and any of the forms—dotted, solid, dashed, or double. The margin, which is the space between the border (or the content of the element if it has no border) and the element's neighbor, can be set with the margin properties. The padding, which is the space between the content of an element and its border (or neighbor if it has no border) can be set with the padding properties.

The `background-image` property is used to place an image in the background of an element.

The `<span>` tag provides a way to include an inline style sheet that applies to a range of text that is smaller than a line or a paragraph. The `<div>` tag provides a way to define a section of a document that has its own style properties.

Conflict resolution for property values is a complicated process, using the origin of specifications, their specificity, inheritance, and the relative position of specifications.

## Review Questions

3.1 What is the advantage of document-level style sheets over inline style sheets?

3.2 What is the purpose of external style sheets?

3.3 What attributes are required in a link to an external style sheet?

3.4 What is the format of an inline style sheet?

3.5 What is the format of a document-level style sheet, and where does it appear?

3.6 What is the format of an external style sheet?

3.7 What is the form of comments within the rule list of a document-level style sheet?

3.8 What is the purpose of a style class selector?

3.9 What is the purpose of a generic class?

3.10 Are keyword property values case sensitive or case insensitive?

3.11 Why is a list of font names given as the value of a `font-family` property?

3.12 What are the five generic fonts?

3.13 In what order must property values appear in the list of a `font` property?

3.14 In what ways can text be modified with `text-decoration`?

3.15 How is the `list-style-type` property used with unordered lists?

3.16 What are the possible values of the `list-style-type` property when it is used with ordered lists?

3.17 If you want text to flow around the right side of an image, which value, `right` or `left`, must be assigned to the `float` property of the image?

3.18 Why must background images be chosen with care?

3.19 What are the possible values for the `text-align` property?

3.20 What purpose does the `text-indent` property serve?

3.21 What properties are used to set margins around elements?

3.22 What are the three ways color property values can be specified?

3.23 If you want a background image to be repeated vertically but not horizontally, what value must be set to what property?

3.24 What properties and what values must be used to put a dotted border around a text box, where the border is red and thin on the left and blue and thick on the right?

3.25 What layout information does a `<span>` tag by itself indicate to the browser?

3.26 What is the purpose of the `<div>` tag?

3.27 Which has higher precedence, an `id` selector or a universal selector?

3.28 Which has higher precedence, a user-origin specification or a browser specification?

3.29 If there are two conflicting specifications in a document-level style sheet, which of the two has precedence?

## Exercises

3.1 Create an external style sheet for the chapters of this book.

3.2 Create and test an XHTML document that displays a table of football scores from a collegiate football conference in which the team names have one of the primary colors of their respective schools. The winning scores must appear larger and in a different font than the losing scores. The team names must be in a script font.

3.3 Create and test an XHTML document that includes at least two images and enough text to precede the images, flow around them (one on the left and one on the right), and continue after the last image.

3.4 Create and test an XHTML document that has at least a half page of text and that has a small box of text embedded on the left margin, with the main text flowing around the small box. The embedded text must appear in a smaller font and also must be set in italic.

3.5 Create and test an XHTML document that has six short paragraphs of text that describe various aspects of the state in which you live. You must define three different paragraph styles, `p1`, `p2`, and `p3`. The `p1` style must use left and right margins of 20 pixels, a background color of pink, and a foreground color of blue. The `p2` style must use left and right margins of 30 pixels, a background color of black, and a foreground color of yellow. The `p3` style must use a text indent of 1 centimeter, a background color of green, and a foreground color of white. The first and fourth paragraph must use `p1`, the second and fifth must use `p2`, and the third and sixth must use `p3`.

3.6 Create and test an XHTML document that describes nested ordered lists of cars. The outer list must have three entries: compact, midsize, and sports. Inside each of these three lists there must be two sublists of body styles. The compact and midsize car sublists are two door and four door; the sports car sublists are coupe and convertible. Each body-style sublist must have at least three entries, each of which is the make and

model of a particular car that fits the category. The outer list must use uppercase Roman numerals, the middle lists must use uppercase letters, and the inner lists must use Arabic numerals. The background color for the compact car list must be pink; for the midsize car list it must be blue; for the sports car list, it must be red. All of the styles must be in a document style sheet.

3.7 Rewrite the document of Exercise 3.6 to put all style-sheet information in an external style sheet. Validate your external style sheet with the W3C CSS validation service.

3.8 Rewrite the document of Exercise 3.6 to use inline style sheets only.

3.9 Create and test an XHTML document that contains at least five lines of text from a newspaper story. Every verb in the text must be green, every noun must be blue, and every preposition must be yellow.

3.10 Create and test an XHTML document that describes an unordered list of at least five popular books. The bullet for each book must be a small image of the book's cover. Find the images on the Web.

3.11 Modify the XHTML document, `nested_lists.html` in Section 2.7.2, to make the different levels of lists different colors using a document style sheet.

3.12 Using a document style sheet, modify the XHTML document, `definition.html` in Section 2.7.3, to set the font in the `dt` elements to Courier 12 point font and the `dd` elements to Times Roman 14 point italic font.

CHAPTER

4

# The Basics of JavaScript


4.1 Overview of JavaScript
4.2 Object Orientation and JavaScript
4.3 General Syntactic Characteristics
4.4 Primitives, Operations, and Expressions
4.5 Screen Output and Keyboard Input
4.6 Control Statements
4.7 Object Creation and Modification
4.8 Arrays
4.9 Functions
4.10 An Example
4.11 Constructors
4.12 Pattern Matching Using Regular Expressions
4.13 Another Example
4.14 Errors in Scripts
*Summary • Review Questions • Exercises*


**This chapter takes you on a quick tour of the basics of JavaScript, introducing its most important concepts and constructs, but leaving out many of the details of the language.** Topics discussed include the following: primitive data types and their operators and expressions, screen output and keyboard input, control statements, objects and constructors, arrays, functions, and pattern matching. In spite of this chapter's brevity, if you are an experienced program-

mer, you should be able to learn how to be an effective JavaScript programmer by studying this chapter, along with Chapter 5, "JavaScript and XHTML Documents," and Chapter 6, "Dynamic Documents with JavaScript." More comprehensive descriptions of JavaScript can be found in the numerous books devoted solely to JavaScript.

## 4.1 Overview of JavaScript

This section discusses the origins of JavaScript, a few of its characteristics, and some of its uses. Included are a comparison of JavaScript and Java and a brief introduction to event-driven programming.

### 4.1.1 Origins

JavaScript, which was originally named LiveScript, was developed by Netscape. In late 1995 LiveScript became a joint venture of Netscape and Sun Microsystems and its name was changed to JavaScript. Netscape's JavaScript has gone through extensive evolution, moving from version 1.0 to version 1.5, primarily by adding many new features. A language standard for JavaScript was developed in the late 1990s by the European Computer Manufacturers Association (ECMA) as ECMA-262. This standard has also been approved by the International Standards Organization (ISO) as ISO-16262. The ECMA-262 standard is now in version 3, which corresponds to Netscape's version 1.5 of JavaScript. Microsoft's JavaScript is named JScript. The FireFox 2 (FX2) and Internet Explorer 7 (IE7) browsers both implement languages that conform to ECMA-262 v3. The current standard specification can be found at

```
http://www.ecma-international.org/publications/
standards Ecma-262.htm
```

The official name of the standard language is ECMAScript. Because it is nearly always called JavaScript elsewhere, we will use that term exclusively in this book.

JavaScript can be divided into three parts: the core, client side, and server side. The *core* is the heart of the language, including its operators, expressions, statements, and subprograms. *Client-side* JavaScript is a collection of objects that support control of a browser and interactions with users. For example, with JavaScript, an XHTML document can be made to be responsive to user inputs such as mouse clicks and keyboard use. *Server-side* JavaScript is a collection of objects that make the language useful on a Web server; for example, to support communication with a database management system.

Server-side JavaScript is used far less frequently than client-side JavaScript. Because of this, this book does not cover server-side JavaScript.

Client-side JavaScript is an XHTML-embedded scripting language. We refer to every collection of JavaScript code as a *script*. An XHTML document can include any number of embedded scripts.

### 4.1.2 JavaScript and Java

Although JavaScript's name appears to connote a close relationship with Java, JavaScript and Java are actually very different. One important difference is support for object-oriented programming. Although JavaScript is sometimes said to be an object-oriented language, its object model is quite different from that of Java and C++, as you will see in Section 4.2. In fact, JavaScript does not support the object-oriented software development paradigm.

Java is a strongly typed language. Types are all known at compile time, and operand types are checked for compatibility. Variables in JavaScript need not be declared and are dynamically typed, making compile-time type checking impossible. One more important difference between Java and JavaScript is that objects in Java are static in the sense that their collection of data members and methods is fixed at compile time. JavaScript objects are dynamic—the number of data members and methods of an object can change during execution.

The main similarity between Java and JavaScript is the syntax of their expressions, assignment statements, and control statements.

### 4.1.3 Uses of JavaScript

The original goal of JavaScript was to provide programming capability at both the server and the client ends of a Web connection. Since then, JavaScript has grown into a full-fledged programming language that can be used for a variety of application areas. This book focuses on client-side JavaScript.

Client-side JavaScript can serve as an alternative for some of what is done with server-side programming, in which computational capability resides on the server and is requested by the client. Client-side JavaScript, on the other hand, is embedded in XHTML documents (either physically or logically) and is interpreted by the browser. This transfer of load from the often-overloaded server to the normally underloaded client can obviously benefit all other clients. Client-side JavaScript cannot replace all server-side computing. In particular, while server-side software supports file operations, database access, and networking, client-side JavaScript supports none of these.

JavaScript can be used as an alternative to Java applets.[1] JavaScript has the advantage of being easier to learn and use than Java. Also, Java applets are downloaded separately from the XHTML documents that call them; many JavaScript scripts, however, are an integral part of the XHTML document, so no secondary downloading is necessary. On the other hand, Java applets are far more capable of producing graphics in documents than are JavaScript scripts.

Interactions with users through form elements, such as buttons and menus, can be conveniently described in JavaScript. Because events, such as button clicks and mouse movements, are easily detected with JavaScript, they can be used to trigger computations and provide feedback to the user. For example, when a user moves the mouse curser from a text box, JavaScript can detect that

1. Java applets are discussed in Appendix C.

movement and check the appropriateness of the text box's value (which presumedly was just filled by the user). Even without forms, user interactions are both possible and simple to program. These interactions, which take place in dialog windows, include getting input from the user and allowing the user to make choices through buttons. It is also easy to generate new content in the browser display dynamically.

Another interesting capability of JavaScript was made possible by the development of the Document Object Model (DOM), which allows JavaScript scripts to access and modify the CSS properties and content of any element of a displayed XHTML document, making formally static documents highly dynamic. Various techniques for designing dynamic XHTML documents with JavaScript are discussed in Chapter 6.

### 4.1.4 Event-Driven Computation

Much of what JavaScript scripts typically do is event driven, meaning that the actions often are executed in response to actions of the users of documents, among them mouse clicks and form submissions. This form of computation supports user interactions through the XHTML form elements on the client display. One of the common uses of JavaScript is to check the values provided in forms by users to determine whether the values are sensible. Without client-side checks of such values, form values must be transmitted to the server for processing without any prior reality checks. The program or script on the server that processes the form data must check for invalid input data. When invalid data is found, the server must transmit that information back to the browser, which then must ask the user to resubmit corrected input. It is obviously more efficient to perform input data checks and carry on this user dialog entirely on the client. It saves both server time and Internet time. Note, however, that validity checking on form data is often also performed on the server, in part because client-side validity checking can be subverted by an unscrupulous user. For certain form data, validity is very important. One example is if the data is to be put in a database where invalid data could corrupt the database.

The mechanics of event-driven computation in JavaScript are discussed in detail in Chapter 5.

### 4.1.5 Browsers and XHTML/JavaScript Documents

If an XHTML document does not include embedded scripts, the browser reads the lines of the document and renders its window according to the tags, attributes, and content it finds. When a JavaScript script is encountered in the document, the browser uses its JavaScript interpreter to "execute" the script. When the end of the script is reached, the browser goes back to reading the XHTML document and displaying its content.

JavaScript scripts can appear in either part of an XHTML document, the head or the body, depending on the purpose of the script. Scripts that produce content only when requested or that react to user interactions are placed in the

head of the document. Generally, this means function definitions and code associated with form elements such as buttons. On the other hand, scripts that are to be interpreted just once, when the interpreter finds them, are placed in the document body. Accordingly, the interpreter notes the existence of scripts that appear in the head of a document, but it does not interpret them while processing the head. Scripts that are found in the body of a document are interpreted as they are found.

## 4.2 Object Orientation and JavaScript

As stated previously, JavaScript is not an object-oriented programming language. Rather, it is an object-based language. JavaScript does not have classes. Its objects serve both as objects and as models of objects. Without classes, JavaScript cannot have class-based inheritance, as is supported in object-oriented languages such as C++ and Java. It does, however, support a technique that can be used to simulate some of the aspects of inheritance. This is done with the prototype object; thus, this form of inheritance is called *prototype-based inheritance*. Prototype-based inheritance is not discussed in this book.

Without class-based inheritance, JavaScript cannot support polymorphism. A polymorphic variable can reference related objects of different classes within the same class hierarchy. A method call through such a polymorphic variable can be dynamically bound to the method in the object's class.[2]

Despite the fact that JavaScript is not an object-oriented language, much of its design is rooted in the concepts and approaches used in object-oriented programming. Specifically, client-side JavaScript deals in large part with documents and document elements, which are modeled with objects.

### 4.2.1 JavaScript Objects

In JavaScript, objects are collections of properties, which correspond to the members of classes in Java and C++. Each property is either a data property or a function or method property. Data properties appear in two categories: primitive values and references to other objects. (In JavaScript, variables that refer to objects are often called *objects* rather than *references*.) Sometimes we will refer to the data properties simply as *properties;* we often refer to the method properties simply as *methods* or *functions*. We prefer to call subprograms that are called through objects methods and subprograms that are not called through objects functions.

The more general category of object properties is other objects. JavaScript uses nonobject types for some of its simplest data types; these types are called *primitives*. Primitives are used because they often can be implemented directly in hardware, resulting in faster operations on their values (faster than if they

2. This is often called *dynamic binding*. It is an essential part of full support for object-oriented programming in a language.

were treated as objects). Primitives are like the simple scalar variables of non-object-oriented languages such as C. C++, Java, and JavaScript all have both primitives and objects; JavaScript's primitives are described in Section 4.4.

All objects in a JavaScript program are indirectly accessed through variables. Such a variable is like a reference in Java. All primitive values in JavaScript are accessed directly—these are like the scalar types in Java and C++. These are often called *value types*. The properties of an object are referenced by attaching the name of the property to the variable that references the object. For example, if `myCar` is a variable that is referencing an object that has the property `engine`, the `engine` property can be referenced with `myCar.engine`.

The root object in JavaScript is `Object`. It is the ancestor, through prototype inheritance, of all objects. `Object` is the most generic of all objects, having some methods but no data properties. All other objects are specializations of `Object`, and all inherit its methods (although they are often overriden).[3]

A JavaScript object appears, both internally and externally, as a list of property/value pairs. The properties are names; the values are data values or functions. All functions are objects and are referenced through variables. The collection of properties of a JavaScript object is dynamic—properties can be added or deleted at any time.

Every object is characterized by its collection of properties, although objects do not have types in any formal sense. Recall that `Object` is characterized by having no properties. Futher discussion of objects appears in Sections 4.7 and 4.11.

## 4.3 General Syntactic Characteristics

In this book all JavaScript scripts are embedded, either directly or indirectly, in XHTML documents. Scripts can appear directly as the content of a `<script>` tag. The `type` attribute of `<script>` must be set to `"text/javascript"`. The JavaScript script can be indirectly embedded in an XHTML document using the `src` attribute of a `<script>` tag, whose value is the name of a file that contains the script. For example:

```
<script type = "text/javascript" src = "tst_number.js" >
</script>
```

Notice that the `script` element requires the closing tag, even though it has no content when the `src` attribute is included. The indirect method of embedding JavaScript in XHTML documents has the advantage of hiding the script from the browser user. It also avoids the problem of hiding scripts from older browsers, which is discussed later in this section. Furthermore, it is good to separate the computation provided by JavaScript from the layout and presentation pro-

3. It sounds like a contradiction when we say that all objects inherit methods from `Object`, although we said earlier that `Object` has no properties. The answer to this paradox lies in the design of prototype inheritance in JavaScript. Every object has a prototype object associated with it. It is `Object`'s prototype object that defines the methods that are inherited by all other objects.

vided by XHTML and CSS, respectively. On the other hand, there are many situations when a small amount of JavaScript code is embedded in an XHTML document. Furthermore, some documents have a large number of places where JavaScript code is embedded. Therefore, it is often inconvenient and cumbersome to place all JavaScript code in a separate file.

In JavaScript, identifiers, or names, are similar to those of other common programming languages. They must begin with a letter, an underscore (_), or a dollar sign ($).[4] Subsequent characters may be letters, underscores, dollar signs, or digits. There is no length limitation for identifiers. As is the case with most C-based languages, the letters in a variable name in JavaScript are case sensitive, meaning that `FRIZZY`, `Frizzy`, `FrIzZy`, `frizzy`, and `frizZy` are all distinct names. However, by convention, programmer-defined variable names do not include uppercase letters.

JavaScript has 25 reserved words, which are listed in Table 4.1.

**Table 4.1** JavaScript reserved words

| | | | | |
|---|---|---|---|---|
| `break` | `delete` | `function` | `return` | `typeof` |
| `case` | `do` | `if` | `switch` | `var` |
| `catch` | `else` | `in` | `this` | `void` |
| `continue` | `finally` | `instanceof` | `throw` | `while` |
| `default` | `for` | `new` | `try` | `with` |

Besides its reserved words, another collection of words is reserved for future use in JavaScript—these can be found at the ECMA Web site. In addition, JavaScript has a large collection of predefined words, including `alert`, `open`, `java`, and `self`.

JavaScript has two forms of comments, both of which are used in other languages. First, whenever two adjacent slashes (`//`) appear on a line, the rest of the line is considered a comment. Second, both single- and multiple-line comments can be written using `/*` to introduce the comment and `*/` to terminate it.

There are two issues regarding embedding JavaScript in XHTML documents. First, there are some browsers still in use that recognize the `<script>` tag but do not have JavaScript interpreters. These browsers simply ignore the contents of the script element and cause no problems. Second, there are still a few browsers in use that are so old they do not recognize the `<script>` tag. Such a browser would display the contents of the script element as if it were just text. It has been customary to enclose the contents of all script elements in XHTML comments to avoid this problem. Because there are so few browsers that do not recognize the `<script>` tag, we believe this is no longer a problem.

4. Dollar signs are not intended to be used by user-written scripts, although it is legal.

However, the XHTML validator also has a problem with embedded JavaScript. When the embedded JavaScript happens to include recognizable tags—for example `<br />` tags in the output of the JavaScript—they often cause validation errors. Therefore, we still enclose embedded JavaScript in XHTML comments.

The XHTML comment introduction (`<!--`) works as a hiding prelude to JavaScript code. However, the syntax for closing a comment that encloses JavaScript code is special. It is the usual XHTML comment closer, but it must be on its own line and must be preceded by two slashes (which makes it a JavaScript comment). The following XHTML comment form hides the enclosed script from browsers that do not have JavaScript interpreters, but makes it visible to browsers that do support JavaScript:

```
<!--
-- JavaScript script --
// -->
```

There are other problems with putting embedded JavaScript in comments in XHTML documents. These are discussed in Chapter 6. The best solution to all of these problems is to put all JavaScript scripts of significant size in separate files.

The use of semicolons in JavaScript is unusual. The JavaScript interpreter tries to make semicolons unnecessary, but it does not always work. When the end of a line coincides with what could be the end of a statement, the interpreter effectively inserts a semicolon there. But this can lead to problems. For example, consider the following:

```
return
x;
```

The interpreter puts a semicolon after `return`, making `x` an illegal orphan. The safest way to organize JavaScript statements is to put each on its own line whenever possible and terminate each statement with a semicolon. If a statement does not fit on a line, be careful to break the statement at a place that will ensure that the first line does not have the form of a complete statement.

The following is a complete, but trivial XHTML document that simply greets the client who requests it. There is but one line of JavaScript in the document, the call to `write` through the `document` object to display the message.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- hello.html
     A trivial hello world example of XHTML/JavaScript
     -->
```

```
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Hello world </title>
  </head>
  <body>
    <script type = "text/javascript">
      <!--
      document.write("Hello, fellow Web programmers!");
      // -->
    </script>
  </body>
</html>
```

## 4.4 Primitives, Operations, and Expressions

The primitive data types, operations, and expressions of JavaScript are similar to those of other common programming languages. Therefore, our discussion of them is brief.

### 4.4.1 Primitive Types

JavaScript has five primitive types: Number, String, Boolean, Undefined, and Null.[5] All primitive values have one of these types. JavaScript includes predefined objects that are closely related to the Number, String, and Boolean types, named `Number`, `String`, and `Boolean`. Is this confusing yet? These objects are called *wrapper objects*. Each contains a property that stores a value of the corresponding primitive type. The purpose of the wrapper objects is to provide properties and methods that are convenient for use with values of the primitive types. In the case of `Number`, the properties are more useful; in the case of `String`, the methods are more useful. Because JavaScript coerces values between the Number type and `Number` objects and between the String type and `String` objects, the methods of `Number` and `String` can be used on variables of the corresponding primitive types. In fact, in most cases you can simply treat Number and String type values as if they were objects.

The difference between primitives and objects is shown in the following example. Suppose that `prim` is a primitive variable with the value `17` and `obj` is a `Number` object whose property value is `17`. Figure 4.1 shows how `prim` and `obj` are stored.

5. Undefined and Null are often called *trivial* types, for reasons that will be obvious when these types are discussed in Section 4.4.3.

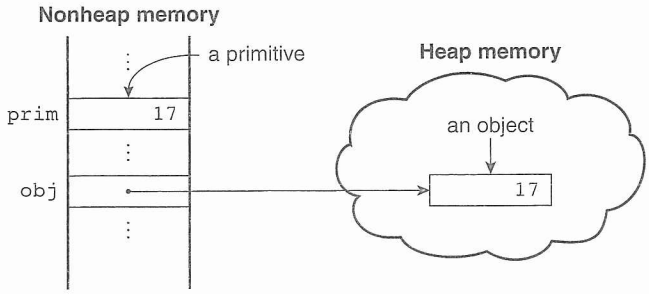


**Figure 4.1** Primitives and objects

### 4.4.2 Numeric and String Literals

All numeric literals are values of type Number. The numeric values of JavaScript are represented internally in double-precision floating-point form. Because of this single numeric data type, numeric values in JavaScript are often called *numbers*. Literal numbers in a script can have the forms of either integer or floating-point values. Integer literals are strings of digits. Floating-point literals can have decimal points or exponents or both. Exponents are specified with an uppercase or lowercase `e` and a possibly signed integer literal. The following are legal numeric literals:

```
72    7.2    .72    72.    7E2    7e2    .7e2    7.e2    7.2E-2
```

Integer literals can be written in hexadecimal form by preceding their first digit with either `0x` or `0X` (the first character is zero, not "oh").

A string literal is a sequence of zero or more characters delimited by either single quotes (`'`) or double quotes (`"`). String literals can include characters specified with escape sequences, such as `\n` and `\t`. If you want an actual single-quote character in a string literal that is delimited by single quotes, the embedded single quote must be preceded by a backslash:

```
'You\'re the most freckly person I\'ve ever met'
```

A double quote can be embedded in a double-quoted string literal by preceding it with a backslash. An actual backslash character in any string literal must be itself backslashed, as in the following example:

```
"D:\\bookfiles"
```

There is no difference between single-quoted and double-quoted literal strings. The null string (one with no characters) can be denoted with either `''` or `""`. All string literals are primitive values.

### 4.4.3 Other Primitive Types

The only value of type Null is the reserved word `null`, which indicates no value. A variable is `null` if it has not been explicitly declared or assigned a value.

If an attempt is made to use the value of a variable whose value is null, it will cause a runtime error.

The only value of type Undefined is `undefined`. Unlike `null`, there is no reserved word `undefined`. If a variable has been explicitly declared, but not assigned a value, it has the value `undefined`. If the value of an undefined variable is displayed, the word `"undefined"` is displayed.

The only values of type Boolean are `true` and `false`. These values are usually computed as the result of evaluating a relational or Boolean expression (see Section 4.6.1). The existence of both the Boolean primitive type and the `Boolean` object can lead to some confusion (also discussed in Section 4.6.1).

### 4.4.4 Declaring Variables

One of the characteristics of JavaScript that sets it apart from most other common programming languages is that it is dynamically typed. This means that a variable can be used for anything. Variables are not typed, values are. A variable can have the value of any primitive type, or it can be a reference to any object. The type of the value of a particular appearance of a variable in a program is determined by the interpreter. In many cases, the interpreter converts the type of a value to whatever is needed for the context in which it appears.

A variable can be declared either by assigning it a value, in which case the interpreter implicitly declares it to be a variable, or by listing it in a declaration statement that begins with the reserved word `var`. Initial values can be included in a `var` declaration, as with some of the variables in the following declaration:

```
var counter,
    index,
    pi = 3.14159265,
    quarterback = "Elway",
    stop_flag = true;
```

We recommend that all variables be explicitly declared.

As stated previously, a variable that has been declared, but not assigned a value, has the value `undefined`.

### 4.4.5 Numeric Operators

JavaScript has the typical collection of numeric operators. These are the binary operators + for addition, - for subtraction, * for multiplication, / for division, and % for modulus. The unary operators are plus (+), negate (-), decrement (--), and increment (++). The increment and decrement operators can be either prefix or postfix.[6] As with other languages that have the increment and decrement unary operators, the prefix and postfix uses are not always equivalent. Consider an expression consisting of a single variable and one of these opera-

6. *Prefix* means that the operator precedes its operand; *postfix* means that the operator follows its operand.

tors. If the operator precedes the variable, the value of the variable is changed and the expression evaluates to the new value. If the operator follows the variable, the expression evaluates to the current value of the variable, and then the value of the variable is changed. For example, if the variable `a` has the value `7`, the value of the following expression is `24`:

```
(++a) * 3
```

But the value of the following expression is `21`:

```
(a++) * 3
```

In both cases, `a` is set to `8`.

All numeric operations are done in double-precision floating point.

The *precedence rules* of a language specify which operator is evaluated first when two operators with different precedence are adjacent in an expression. Adjacent operators are separated by a single operand. For example, in the following * and + are adjacent:

```
a * b + 1
```

The *associativity rules* of a language specify which operator is evaluated first when two operators with the same precedence are adjacent in an expression. The precedence and associativity of the numeric operators of JavaScript are given in Table 4.2.

**Table 4.2** Precedence and associativity of the numeric operators

| Operator | Associativity |
|---|---|
| ++, --, unary -, unary + | Right (though it is irrelevant) |
| *, /, % | Left |
| Binary +, binary - | Left |

The first operators listed have the highest precedence.

As examples of operator precedence and associativity, consider the following code:

```
var a = 2,
    b = 4,
    c,
    d;
c = 3 + a * b;
// * is first, so c is now 11 (not 24)
d = b / a / 2;
// / associates left, so d is now 1 (not 4)
```

Parentheses can be used to force any desired precedence. For example, the addition will be done before the multiplication in the following expression:

```
(a + b) * c
```

### 4.4.6 The Math Object

The `Math` object provides a collection of properties of `Number` objects and methods that operate on `Number` objects. The `Math` object has methods for the trigonometric functions, such as `sin` (for sine) and `cos` (for cosine), as well as for other commonly used mathematical operations. Among these are `floor`, to truncate a number; `round`, to round a number; and `max`, to return the largest of two given numbers. The `floor` and `round` methods are used in the example script in Section 4.10. All of the `Math` methods are referenced through the `Math` object, as in `Math.sin(x)`.

### 4.4.7 The Number Object

The `Number` object includes a collection of useful properties that have constant values. Table 4.3 lists the properties of `Number`. These properties are referenced through `Number`. For example:

```
Number.MIN_VALUE
```

**Table 4.3** Properties of `Number`

| Property | Meaning |
|---|---|
| `MAX_VALUE` | Largest representable number |
| `MIN_VALUE` | Smallest representable number |
| `NaN` | Not a number |
| `POSITIVE_INFINITY` | Special value to represent infinity |
| `NEGATIVE_INFINITY` | Special value to represent negative infinity |
| `PI` | The value of $\pi$ |

Any arithmetic operation that results in an error (for example, division by zero) or that produces a value that cannot be represented as a double-precision floating-point number, such as one that is too large (overflow), returns the value "not a number," which is displayed as `NaN`. If `NaN` is compared for equality against any number, the comparison fails. Surprisingly, in a comparison, `NaN` is not equal to itself. To determine whether a variable has the `NaN` value, the predefined predicate function `isNaN()` must be used. For example, if the variable `a` has the `NaN` value, `isNaN(a)` returns `true`.

The `Number` object has a method, `toString`, which it inherits from `Object` but overrides. The `toString` method converts the number through which it is called to a string. Because numeric primitives and `Number` objects are always coerced to the other when necessary, `toString` can be called through a numeric primitive. For example:

```
var price = 427,
    str_price;
...
str_price = price.toString();
```

### 4.4.8 The String Catenation Operator

JavaScript strings are not stored or treated as arrays of characters; rather, they are unit scalar values. String catenation is specified with the operator denoted by a plus sign (+). For example, if the value of `first` is `"Freddie"`, the value of the following expression is `"Freddie Freeloader"`:

```
first + " Freeloader"
```

### 4.4.9 Implicit Type Conversions

The JavaScript interpreter performs several different implicit type conversions. Such conversions are called *coercions*. In general, when a value of one type is used in a position that requires a value of a different type, JavaScript attempts to convert the value to the type that is required. The most common examples of these conversions involve primitive string and number values.

If either operand of a + operator is a string, the operator is interpreted as a string catenation operator. If the other operand is not a string, it is coerced to a string. For example, consider the following expression:

```
"August " + 1977
```

In this expression, because the left operand is a string, the operator is considered to be a catenation operator. This forces string context on the right operand, so the right operand is implicitly converted to a string. Therefore, this expression evaluates to the following:

```
"August 1997"
```

The number `1977` in the following expression is also coerced to a string:

```
1977 + "August"
```

Now consider the following expression:

```
7 * "3"
```

In this expression the operator is one that is only used with numbers. This forces numeric context on the right operand. Therefore, JavaScript attempts to convert it to a number. In this example the conversion succeeds, and the value of

this expression is 21. If the second operand were a string that could not be converted to a number, such as "August", the conversion would produce NaN, which would be the value of the expression.

When used as a number, null is 0. Unlike C and C++, however, null is not the same as 0. When used as a number, undefined is interpreted as NaN (see Section 4.4.7).

When interpreted as a Boolean (put in Boolean context), the number 0 is false and all other numbers are true. When a string is interpreted as a Boolean, the empty string is false and all other strings are true. If the special value, NaN, is initerpreted as a Boolean, it is false. If undefined is used as a Boolean, it is false. When interpreted as a Boolean, null is false. When interpreted as a number, true has the value 1 and false has the value 0.

As we will see in Section 4.6.1, the relational operators also can cause implicit type conversions.

### 4.4.10 Explicit Type Conversions

There are several different ways to force type conversions, primarily between strings and numbers. Strings that contain numbers can be converted to numbers with the String constructor, as in the following:

```
var str_value = String(value);
```

This conversion could also be done with the toString method, which has the advantage that it can be given a parameter to specify the base of the resulting number (although the base of the number to be converted is taken to be decimal). For example:

```
var num = 6;
var str_value = num.toString();
var str_value_binary = num.toString(2);
```

In the first conversion, the result is "6"; in the second, it is "110".

A number also can be converted to a string by catenating it with the empty string.

Strings can be explicitly converted to numbers in a variety of ways. The Number constructor can be used, as in the following:

```
var number = Number(aString);
```

The same conversion could be specified by subtracting zero from the string, as in the following:

```
var number = aString - 0;
```

Both of these conversions have the following restriction: The number in the string cannot be followed by any character except a space. For example, if the number happens to be followed by a comma, the conversion will not work. JavaScript has two predefined string functions that do not have this problem. These two, parseInt and parseFloat, are not String methods,

so they are not called through `String` objects; however, they operate on the strings given as parameters. The `parseInt` function searches the string for an integer literal. If one is found at the beginning of the string, it is converted to a number and returned. If the string does not begin with a valid integer literal, `NaN` is returned. The `parseFloat` function is similar to `parseInt`, but it searches for a floating-point literal, which could have a decimal point or an exponent or both. In both `parseInt` and `parseFloat`, the numeric literal could be followed by any nondigit character without causing any problems.

Because of the coercions JavaScript normally does, as discussed in Section 4.4.9, `parseInt` and `parseFloat` are not often needed.

### 4.4.11 `String` Properties and Methods

Because JavaScript coerces primitive string values to and from `String` objects when necessary, the differences between the `String` object and the String type have little effect on scripts. `String` methods can always be used through String primitive values, as if the values were objects. The `String` object includes one property, `length`, and a large collection of methods.

The number of characters in a string is stored in the `length` property as follows:

```
var str = "George";
var len = str.length;
```

In this code, `len` is set to the number of characters in `str`, `6`. In the expression `str.length`, `str` is a primitive variable, but we treated it as if it were an object (referencing one of its properties). In fact, when `str` is used with the `length` property, JavaScript implicitly builds a temporary `String` object with a property whose value is that of the primitive variable. After the second statement is executed, the temporary `String` object is discarded.

A few of the most commonly used `String` methods are shown in Table 4.4.

**Table 4.4** `String` methods

| Method | Parameters | Result |
|---|---|---|
| `charAt` | A number | Returns the character in the `String` object that is at the specified position |
| `indexOf` | One-character string | Returns the position in the `String` object of the parameter |
| `substring` | Two numbers | Returns the substring of the `String` object from the first parameter position to the second |
| `toLowerCase` | None | Converts any uppercase letters in the string to lowercase |
| `toUpperCase` | None | Converts any lowercase letters in the string to uppercase |

Note that for the `String` methods, character positions start at zero.

For example, suppose `str` has been defined as follows:

```
var str = "George";
```

The following expressions have the shown values:

```
str.charAt(2)  is 'o'
str.indexOf('r')  is 3
str.substring(2, 4)  is 'org'
str.toLowerCase()  is 'george'
```

Several `String` methods associated with pattern matching are described in Section 4.12.

### 4.4.12 The typeof Operator

The `typeof` operator returns the type of its single operand. This is quite useful in some circumstances in a script. `typeof` evaluates to `"number"`, `"string"`, or `"boolean"` if the operand is of primitive type Number, String, or Boolean, respectively. If the operand is an object or `null`, `typeof` evaluates to `"object"`. This illustrates a fundamental characteristic of JavaScript—objects do not have types. If the operand is a variable that has not been assigned a value, `typeof` evaluates to `"undefined"`, reflecting the fact that variables themselves are not typed. Notice that the `typeof` operator always returns a string. The operand for `typeof` can be placed in parentheses, making it appear to be a function. Therefore, `typeof x` and `typeof(x)` are equivalent.

### 4.4.13 Assignment Statements

The assignment statement in JavaScript is exactly like the assignment statement in other common C-based programming languages. There is a simple assignment operator, denoted by =, and a host of compound assignment operators, such as += and /=. For example, the statement

```
a += 7;
```

means the same as the following:

```
a = a + 7;
```

When considering assignment statements, it is necessary to remember that JavaScript has two kinds of values—primitives and objects. A variable can refer to a primitive value, such as the number 17, or an object, as shown in Figure 4.1. Objects are allocated on the heap, and variables that refer to them are essentially reference variables. When used to refer to an object, a variable stores an address only. Therefore, assigning the address of an object to a variable is fundamentally different from assigning a primitive value to a variable.

### 4.4.14 The Date Object

There are occasions when information about the current date and time is useful in a program. Likewise, sometimes it is convenient to be able to create objects that represent a specific date and time and manipulate them. These capabilities are available in JavaScript through the `Date` object and its rich collection of methods. In the following, we describe this object and some of its methods.

A `Date` object is created, naturally, with the `new` operator and the `Date` constructor, which has several forms. Because we focus on uses of the current date and time, we use only the simplest `Date` constructor, which takes no parameters and builds an object with the current date and time for its properties. For example:

```
var today = new Date();
```

The date and time properties of a `Date` object are in two forms, local and Coordinated Universal Time (UTC, which was formerly named Greenwich Mean Time). We only deal with local time in this section.

Table 4.5 shows the methods, along with the descriptions, that retrieve information from a `Date` object.

**Table 4.5** Methods for the `Date` object

| Method | Returns |
|---|---|
| `toLocaleString` | A string of the `Date` information |
| `getDate` | The day of the month |
| `getMonth` | The month of the year, as a number in the range of 0 to 11 |
| `getDay` | The day of the week, as a number in the range of 0 to 6 |
| `getFullYear` | The year |
| `getTime` | The number of milliseconds since January 1, 1970 |
| `getHours` | The number of the hour, as a number in the range of 0 to 23 |
| `getMinutes` | The number of the minute, as a number in the range of 0 to 59 |
| `getSeconds` | The number of the second, as a number in the range of 0 to 59 |
| `getMilliseconds` | The number of the millisecond, as a number in the range of 0 to 999 |

The use of the `Date` object is shown in Section 4.6.

## 4.5 Screen Output and Keyboard Input

A JavaScript script is interpreted when the browser finds the script in the body of the XHTML document. Thus, the normal output screen for JavaScript is the

same as the screen in which the content of the host XHTML document is displayed. JavaScript models the XHTML document with the `Document` object. The window in which the browser displays an XHTML document is modeled with the `Window` object. The `Window` object includes two properties, `document` and `window`. The `document` property refers to the `Document` object. The `window` property is self referential; it refers to the `Window` object.

The `Document` object has several properties and methods. The most interesting and useful of its methods, at least for now, is `write`, which is used to create script output, which is dynamically created XHTML document content.[7] This content is specified in the parameter to `write`. For example, the following produces the screen shown in Figure 4.2:

```
document.write("The result is: ", result, "<br />");
```

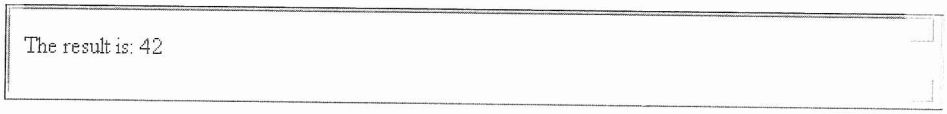


**Figure 4.2** An example of the output of `document.write`

Because `write` is used to create XHTML code, the only useful punctuation in its parameter is in the form of XHTML tags. Therefore, the parameter to `write` often includes `<br />`. The `writeln` method implicitly adds `"\n"` to its parameter, but since browsers ignore line breaks when displaying XHTML, is has no effect on the output.[8]

The parameter to `write` can include any XHTML tags and content. The parameter is simply given to the browser, which treats it exactly like any other part of the XHTML document. The `write` method actually can take any number of parameters. Multiple parameters are concatenated and placed in the output.

As stated previously, the `Window` object is the JavaScript model for the browser window. `Window` includes three methods that create dialog boxes for three specific kinds of user interactions. The default object for JavaScript is the `Window` object currently being displayed, so calls to these methods need not include an object reference.

The `alert` method opens a dialog window and displays its parameter in that window. It also displays an *OK* button. The parameter string to `alert` is not XHTML code; it is plain text. Therefore, the string parameter to `alert` may include `\n` but never should include `<br />`. As an example of an `alert`,

7. The XML Document Object Model does not require XML agents (processors) to implement `write`, although that was likely the intention. Therefore, if an XHTML document is served as XML, some browsers may reject any embedded calls to `write`. However, most XHTML documents are now served as HTML and we believe most browsers will implement `write` for their XML parsers, so we will use `write` in many of our examples.

8. The `writeln` method is useful only if the browser is used to view a non-XHTML document, which is rarely done.

consider the following code, which produces the dialog window shown in Figure 4.3:

```
alert("The sum is:" + sum + "\n");
```

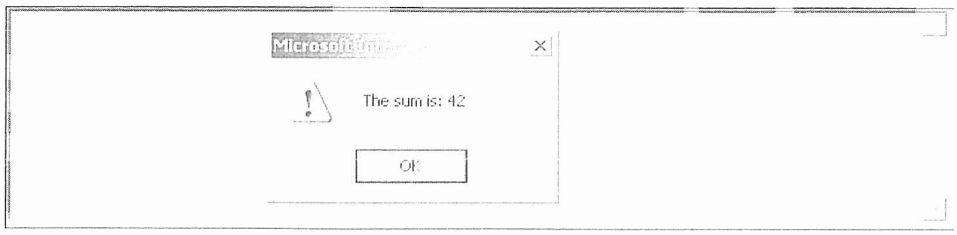


**Figure 4.3** An example of the output of `alert`

The `confirm` method opens a dialog window in which it displays its string parameter, along with two buttons, *OK* and *Cancel*. `confirm` returns a Boolean value that indicates the user's button input: `true` for *OK* and `false` for *Cancel*. This method is often used to offer the user the choice of continuing some process. For example, the following statement produces the screen shown in Figure 4.4:

```
var question =
    confirm("Do you want to continue this download?");
```

After the user presses one of the buttons in the `confirm` dialog window, the script can test the variable, `question`, and react accordingly.

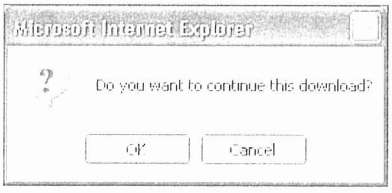


**Figure 4.4** An example of the output of `confirm`

The `prompt` method creates a dialog window that contains a text box. The text box is used to collect a string of input from the user, which `prompt` returns as its value. The window also includes two buttons, *OK* and *Cancel*. `prompt` takes two parameters: the string that prompts the user for input and a default string in case the user does not type a string before pressing one of the two buttons. In many cases, an empty string is used for the default input. Consider the following example:

```
name = prompt("What is your name?", "");
```

Figure 4.5 shows the screen created by this call to `prompt`.

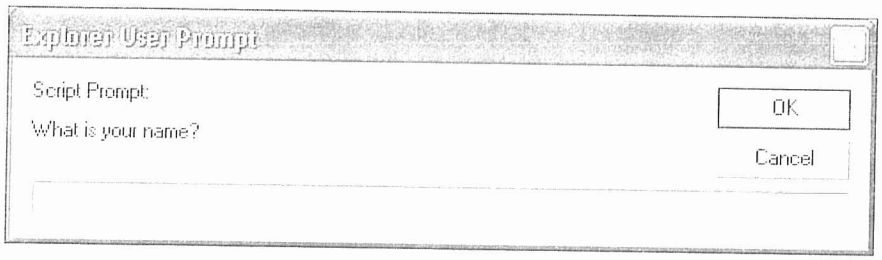


**Figure 4.5** An example of the output of `prompt`

`alert`, `prompt`, and `confirm` cause the browser to wait for a user response. In the case of `alert`, the *OK* button must be pressed for the JavaScript interpreter to continue. The `prompt` and `confirm` methods wait for either *OK* or *Cancel* to be pressed.

The following example XHTML and JavaScript files, `roots.html`, and `roots.js`, illustrate some of the JavaScript features described so far. The JavaScript script gets the coefficients of a quadratic equation from the user with `prompt` and computes and displays the real roots of the given equation. If the roots of the equation are not real, the value `NaN` is displayed. This value comes from the `sqrt` function, which returns `NaN` when given a negative parameter. This corresponds mathematically to the equation not having real roots.

```
<?xml version = "1.0"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- roots.html
     A document for roots.js
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> roots.html </title>
  </head>
  <body>
    <script type = "text/javascript"  src = "roots.js" >
    </script>
  </body>
</html>
```

```
// roots.js
//   Compute the real roots of a given quadratic
//   equation. If the roots are imaginary, this script
//   displays NaN, because that is what results from
//   taking the square root of a negative number

// Get the coefficients of the equation from the user
var a = prompt("What is the value of 'a'? \n", "");
var b = prompt("What is the value of 'b'? \n", "");
var c = prompt("What is the value of 'c'? \n", "");

// Compute the square root and denominator of the result
var root_part = Math.sqrt(b * b - 4.0 * a * c);
var denom = 2.0 * a;

// Compute and display the two roots
var root1 = (-b + root_part) / denom;
var root2 = (-b - root_part) / denom;
document.write("The first root is: ", root1, "<br />");
document.write("The second root is: ", root2, "<br />");
```

## 4.6 Control Statements

This section introduces the flow-control statements of JavaScript. Before discussing the control statements, we must describe the control expressions, which provide the basis for controlling the order of execution of statements. Once again, the similarity of these JavaScript constructs to their counterparts in Java and C++ makes them easy to learn for those who are familiar with one of those languages.

Control statements often require some syntactic container for sequences of statements whose execution they are meant to control. In JavaScript, that container is the compound statement. A *compound statement* in JavaScript is a sequence of statements delimited by braces. A *control construct* is a control statement and the statement or compound statement whose execution it controls.

Unlike several related languages, JavaScript does not allow compound statements to create local variables. If a variable is declared in a compound statement, access to it is not confined to that compound statement. Such a variable is visible in the whole XHTML document.[9] Local variables are discussed in Section 4.9.2.

9. The exception to this rule is if the variable is declared in a function.

### 4.6.1 Control Expressions

The expressions upon which statement flow control can be based include primitive values, relational expressions, and compound expressions. The result of evaluating a control expression is one of the Boolean values `true` or `false`. If the value of a control expression is a string, it is interpreted as `true` unless it is either the empty string (`""`) or a zero string (`"0"`). If the value is a number, it is `true` unless it is zero (`0`).

A relational expression has two operands and one relational operator. Table 4.6 lists the relational operators.

**Table 4.6** Relational operators

| Operation | Operator |
|---|---|
| Is equal to | == |
| Is not equal to | != |
| Is less than | < |
| Is greater than | > |
| Is less than or equal to | <= |
| Is greater than or equal to | >= |
| Is strictly equal to | === |
| Is strictly not equal to | !== |

If the two operands are not of the same type and the operator is neither `===` nor `!==`, JavaScript will attempt to convert the operands to a single type. In the case in which one operand is a string and the other is a number, JavaScript attempts to convert the string to a number. If one operand is Boolean and the other is not, the Boolean value is converted to a number (`1` for `true`, `0` for `false`).

The last two operators in Table 4.6 disallow type conversion of either operand. Thus, the expression `"3" === 3` evaluates to `false`, while `"3" == 3` evaluates to `true`.

Comparisons of variables that reference objects are rarely useful. If `a` and `b` reference different objects, `a == b` is never true, even if the objects have identical properties. `a == b` is true only if `a` and `b` reference the same object.

JavaScript has operators for the AND, OR, and NOT Boolean operations. These are `&&` (AND), `||` (OR), and `!` (NOT). Both `&&` and `||` are short-circuit operators, as they are in Java and C++. This means that if the value of the first operand of either `||` or `&&` determines the value of the expression, the second operand is not evaluated, and the Boolean operator does nothing. JavaScript also has bitwise operators, but they are not discussed in this book.

The properties of the object `Boolean` must not be confused with the primitive values `true` and `false`. If a `Boolean` object is used as a conditional expression, it evaluates to `true` if it has any value other than `null` or `undefined`. The `Boolean` object has a method, `toString`, which it inherits from `Object`, that converts the value of the object through which it is called to one of the strings `"true"` or `"false"`.

The precedence and associativity of all operators discussed so far in this chapter are shown in Table 4.7.

**Table 4.7** Operator precedence and associativity

| Operators | Associativity |
|---|---|
| ++, --, unary - | Right |
| *, /, % | Left |
| +, - | Left |
| >, <, >= ,<= | Left |
| ==, != | Left |
| ===, !== | Left |
| && | Left |
| \|\| | Left |
| =, +=, -=, *=, /=, &&=, \|\|=, %= | Right |

Highest-precedence operators are listed first.

### 4.6.2 Selection Statements

The selection statements (`if-then` and `if-then-else`) of JavaScript are similar to those of the common programming languages. Either single statements or compound statements can be selected. For example:

```
if (a > b)
    document.write("a is greater than b <br />");
else {
    a = b;
    document.write("a was not greater than b <br />",
                   "Now they are equal <br />");
}
```

### 4.6.3 The switch Statement

JavaScript has a `switch` statement that is similar to that of C. The form of this construct follows:

```
switch (expression) {
    case value_1:
        // statement(s)
    case value_2:
        // statement(s)
    ...
    [default:
        // statement(s)]
}
```

In any `case` segment, the statement(s) can be either a statement sequence or a compound statement.

The semantics of a `switch` construct are as follows: The expression is evaluated when the `switch` statement is reached in execution. The value is compared to the values in the cases in the construct (those values that immediately follow the `case` reserved words). If one matches, control is transferred to the statements immediately following that case value. Execution then continues through the remainder of the construct. In the great majority of situations, it is intended that only the statements in one case be executed in each execution of the construct. To implement this, a `break` statement appears as the last statement in each sequence of statements following a case. The `break` statement is exactly like the `break` statement in Java and C++. It transfers control out of the compound statement in which it appears.

The control expression of a `switch` statement could evaluate to a number, a string, or a Boolean value. Case labels also can be numbers, strings, or Booleans, and different case values can be of different types. Consider the following script, which includes a `switch` construct. The XHTML file that includes this script is very simple and thus is not shown.

```
// borders2.js
//   An example of a switch statement for table border
//   size selection

var bordersize;
bordersize = prompt("Select a table border size \n" +
                    "0 (no border) \n" +
                    "1 (1 pixel border) \n" +
                    "4 (4 pixel border) \n" +
                    "8 (8 pixel border) \n");
```

```
switch (bordersize) {
  case "0": document.write("<table>");
            break;
  case "1": document.write("<table border = '1'>");
            break;
  case "4": document.write("<table border = '4'>");
            break;
  case "8": document.write("<table border = '8'>");
            break;
  default:  document.write("Error - invalid choice: ",
                           bordersize, "<br />");
}

document.write("<caption> 2006 NFL Divisional",
               " Winners </caption>");
document.write("<tr>",
               "<th />",
               "<th> American Conference </th>",
               "<th> National Conference </th>",
               "</tr>",
               "<tr>",
               "<th> East </th>",
               "<td> New England Patriots </td>",
               "<td> Philadelphia Eagles </td>",
               "</tr>",
               "<tr>",
               "<th> North </th>",
               "<td> Baltimore Ravens </td>",
               "<td> Chicago Bears </td>",
               "</tr>",
               "<tr>",
               "<th> West </th>",
               "<td> San Diego Chargers </td>",
               "<td> Seattle Seahawks </td>",
               "</tr>",
               "<tr>",
               "<th> South </th>",
               "<td> Indianapolis Colts </td>",
               "<td> New Orleans Saints </td>",
               "</tr>",
               "</table>");
```

The entire table element is produced with `write`. Alternatively, we could have given all of the elements for the table, except the `<table>` and `</table>` tags, directly as XHTML in the XHTML document. Because `<table>` is in the content of the script element, the validator would not see it. Therefore, the `</table>` tag would also need to be hidden.

Browser displays of the prompt dialog box and the output of `borders2.js` are shown in Figures 4.6 and 4.7, respectively.

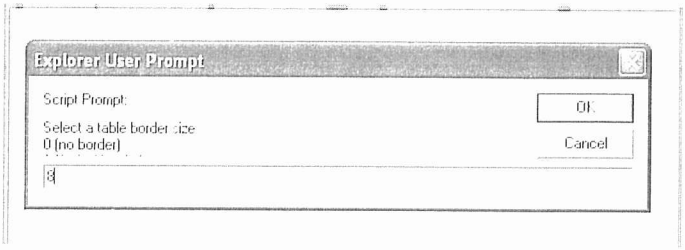


**Figure 4.6** Dialog box from `borders2.js`

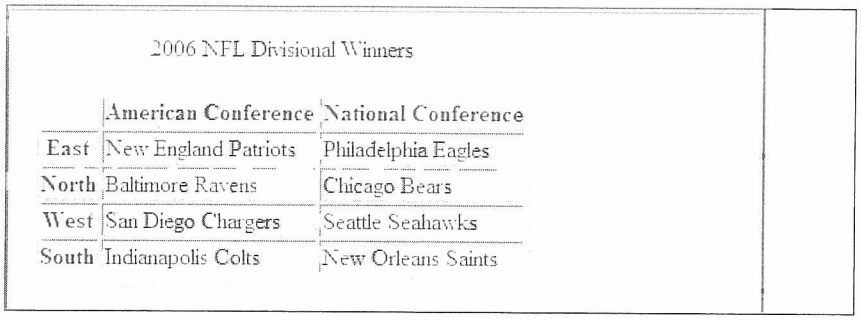


**Figure 4.7** Display of `borders2.js`

### 4.6.4 Loop Statements

The JavaScript `while` and `for` statements are similar to those of Java and C++. The general form of the `while` statement is as follows:

`while` (*control expression*)
  *statement or compound statement*

The general form of the `for` statement is as follows:

`for` (*initial expression; control expression; increment expression*)
  *statement or compound statement*

Both the initial expression and the increment expression can be multiple expressions, separated by commas. The initial expression of a `for` statement can include variable declarations. Such variables are visible in the entire script unless the `for` statement is in a function definition, in which case the variable is visible in the whole function. The following illustrates a simple `for` construct:

```
var sum = 0,
    count;
for (count = 0; count <= 10; count++)
    sum += count;
```

The following is an example that illustrates the `Date` object and a simple `for` loop:

```
// date.js
//   Illustrates the use of the Date object by
//   displaying the parts of a current date and
//   using two Date objects to time a calculation

// Get the current date
var today = new Date();

// Fetch the various parts of the date
var dateString = today.toLocaleString();
var day = today.getDay();
var month = today.getMonth();
var year = today.getFullYear();
var timeMilliseconds = today.getTime();
var hour = today.getHours();
var minute = today.getMinutes();
var second = today.getSeconds();
var millisecond = today.getMilliseconds();

// Display the parts
document.write(
  "Date: " + dateString + "<br />",
  "Day: " + day + "<br />",
  "Month: " + month + "<br />",
  "Year: " + year + "<br />",
  "Time in milliseconds: " + timeMilliseconds + "<br />",
  "Hour: " + hour + "<br />",
  "Minute: " + minute + "<br />",
  "Second: " + second + "<br />",
  "Millisecond: " + millisecond + "<br />");
```

```
// Time a loop
var dum1 = 1.00149265, product = 1;
var start = new Date();

for (var count = 0; count < 10000; count++)
  product = product + 1.000002 * dum1 / 1.00001;

var end = new Date();
var diff = end.getTime() - start.getTime();
document.write("<br />The loop took " + diff +
               " milliseconds <br />");
```

A display of `date.js` is shown in Figure 4.8.

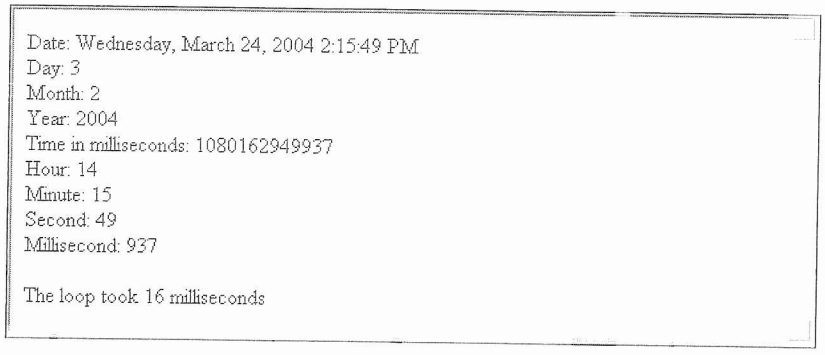


**Figure 4.8** Display of `date.js`

In addition to the `while` and `for` loop statements, JavaScript also has a `do-while` statement, whose form is as follows:

`do` *statement or compound statement*
`while` (*control expression*)

The `do-while` statement is related to the `while` statement, but the test for completion is logically (and physically) at the end rather than the beginning of the loop construct. The body of a `do-while` construct is always executed at least once. The following is an example of a `do-while` construct:

```
do {
    count++;
    sum = sum + (sum * count);
} while count <= 50;
```

JavaScript includes one more loop statement, the `for-in` statement, which is most often used with objects. The `for-in` statement is discussed in Section 4.7.

## 4.7 Object Creation and Modification

Objects are often created with a new expression, which must include a call to a constructor method. The constructor that is called in the new expression creates the properties that characterize the new object. In an object-oriented language such as Java, the new operator creates a particular object, meaning an object with a type and a specific collection of members. Thus, in Java, the constructor initializes members but does not create them. In JavaScript, however, the new operator creates a blank object, or one with no properties. Furthermore, JavaScript objects do not have types. The constructor both creates and initializes the properties.

The following statement creates an object that initially has no properties:

```
var my_object = new Object();
```

In this case, the called constructor is that of `Object`, which endows the new object with no properties, although it does have access to some inherited methods. The variable `my_object` references the new object. Calls to constructors must include parentheses, even if there are no parameters. Constructors are discussed in detail in Section 4.11.

The properties of an object are accessed using dot notation, in which the first word is the object name and the second is the property name. Properties are not actually variables—they are just the names of values. They are used with object variables to access property values. Because properties are not variables, they are never declared.

The number of members of a class in a typical object-oriented language is fixed at compile time. The number of properties in a JavaScript object is dynamic. At any time during interpretation, properties can be added to or deleted from an object. A property for an object is created by assigning a value to that property. Consider the following example:

```
// Create an Object object
var my_car = new Object();
// Create and initialize the make property
my_car.make = "Ford";
// Create and initialize model
my_car.model = "Contour SVT";
```

This code creates a new object, `my_car`, with two properties, `make` and `model`.

There is an abbreviated way to create an object and its properties. For example, the object referenced with `my_car` above could be created with the following statement:

```
var my_car = {make: "Ford", model: "Contour SVT"};
```

Notice that this statement included neither the new operator nor the call to the `Object` constructor.

Because objects can be nested, you can create a new object that is a property of `my_car` with properties of its own as follows:

```
my_car.engine = new Object();
my_car.engine.config = "V6";
my_car.engine.hp = 200;
```

Properties can be accessed in two ways. First, any property can be accessed in the same way it is assigned a value, using the object-dot-property notation. Second, the property names of an object can be accessed as if they were elements of an array, using the property name (as a string literal) as a subscript. For example, consider the following statements:

```
var prop1 = my_car.make;
var prop2 = my_car["make"];
```

After executing these two statements, the variables `prop1` and `prop2` both have the value `"Ford"`.

If an attempt is made to access a property of an object that does not exist, the value `undefined` is used. A property can be deleted with `delete`, as in the following example:

```
delete my_car.model;
```

JavaScript has a loop statement, `for-in`, that is perfect for listing the properties of an object. The form of `for-in` is as follows:

`for (`*identifier* `in` *object*`)`
  *statement or compound statement*

Consider the following example:

```
for (var prop in my_car)
  document.write("Name: ", prop, "; Value: ",
                 my_car[prop], "<br />");
```

The variable, `prop`, takes on the values of the properties of the `my_car` object, one for each iteration. So, this code lists all of the values of the properties of `my_car`.

## 4.8 Arrays

In JavaScript, arrays are objects that have some special functionality. Array elements can be primitive values or references to other objects, including other arrays. JavaScript arrays have dynamic length.

### 4.8.1 Array Object Creation

`Array` objects, unlike most other JavaScript objects, can be created in two distinct ways. The usual way to create any object is with the `new` operator and a call to a constructor. In the case of arrays, the constructor is named `Array`:

```
var my_list = new Array(1, 2, "three", "four");
```

```
var your_list = new Array(100);
```

In the first declaration, an `Array` object of length 4 is created and initialized. Notice that the elements of an array need not have the same type. In the second declaration, a new `Array` object of length 100 is created, without actually creating any elements. Whenever a call to the `Array` constructor has a single parameter, that parameter is taken to be the number of elements, not the initial value of a one-element array.

The second way to create an `Array` object is with a literal array value, which is a list of values enclosed in brackets:

```
var my_list_2 = [1, 2, "three", "four"];
```

The array `my_list_2` has the same values as the `Array` object `my_list` created previously with `new`.

### 4.8.2 Characteristics of Array Objects

The lowest index of every JavaScript array is zero. Array element access is specified with numeric subscript expressions placed in brackets. The length of an array is the highest subscript to which a value has been assigned, plus 1. For example, if `my_list` is an array with four elements and the following statement is executed, the new length of `my_list` will be 48.

```
my_list[47] = 2222;
```

The length of an array is both read- and write-accessible through the `length` property, which is added to every array object by the `Array` constructor. Consequently, the length of an array can be set to whatever you like by assigning the `length` property as follows:

```
my_list.length = 1002;
```

Now, the length of `my_list` is 1002, regardless of what it was previously. Assigning a value to the `length` property can lengthen, shorten, or not affect the array's length (if the value assigned happens to be the same as the previous length of the array).

So, an array can be made to grow either by adding new elements or by setting its `length` property to a larger value. An array can be made to shrink by setting its `length` property to a smaller value.

Only the assigned elements of an array actually occupy space. For example, if it is convenient to use the subscript range of 100 to 150 (but not 0 to 99), an array of length 151 can be created. But if only the elements indexed 100 to 150 are assigned values, the array will require the space of 51 elements, not 151. Remember that the `length` property of an array is not necessarily the number of defined or even allocated elements. For example, the following statement sets the `length` property of `new_list` to `1002`, but `new_list` may have no elements that have values or occupy space:

```
new_list.length = 1002;
```

To support JavaScript's dynamic arrays, all array elements are allocated dynamically from the heap.

The following example, `insert_names.js`, illustrates JavaScript arrays. This script has an array of names, which are in alphabetical order. It uses `prompt` to get new names, one at a time, and inserts them into the existing array. Our approach is to move elements down one at a time, starting at the end of the array, until the correct position for the new name is found. Then the new name is inserted, and the new array is displayed. Notice that each new name causes the array to grow by one element.

```
// insert_names.js
//  This script has an array of names, name_list,
//  whose values are in alphabetical order. New
//  names are input through a prompt. Each new
//  name is inserted into the name_list array,
//  after which the new list is displayed.

// The original list of names
var name_list = new Array("Al", "Betty", "Kasper",
                  "Michael", "Roberto", "Zimbo");
var new_name, index, last;

// Loop to get a new name and insert it
while (new_name =
          prompt("Please type a new name", "")) {
  last = name_list.length - 1;

// Loop to find the place for the new name
  while (last >= 0 && name_list[last] > new_name) {
    name_list[last + 1] = name_list[last];
    last--;
  }

// Insert the new name into its spot in the array
  name_list[last + 1] = new_name;

// Display the new array
  document.write("<p><b>The new name list is:</b> ",
                 "<br />");
  for (index = 0; index < name_list.length; index++)
    document.write(name_list[index], "<br />");
  document.write("</p>");
} //** end of the outer while loop
```

### 4.8.3 Array Methods

`Array` objects have a collection of useful methods, most of which are described here. The `join` method converts all of the elements of an array to strings and catenates them into a single string. If no parameter is provided to `join`, the values in the new string are separated by commas. If a string parameter is provided, it is used as the element separator:

```
var names = new Array["Mary", "Murray",
                      "Murphy", "Max"];
...
var name_string = names.join(" : ");
```

The value of `name_string` is now `"Mary : Murray : Murphy : Max"`.

The `reverse` method does what you would expect: It reverses the order of the elements of the `Array` object through which it is called.

The `sort` method coerces the elements of the array to strings, if they are not already strings, and sorts them alphabetically:

```
names.sort();
```

The value of `names` is now `["Mary", "Max", "Murphy", "Murray"]`. Section 4.9.4 discusses the use of `sort` for different orders and for nonstring elements.

The `concat` method catenates its actual parameters to the end of the `Array` object on which it is called. For example, consider the following code:

```
var names = new Array["Mary", "Murray",
                      "Murphy", "Max"];
...
var new_names = names.concat("Moo", "Meow");
```

The `new_names` array now has length 6, with the elements of `names`, along with `"Moo"` and `"Meow"` as its fifth and sixth elements.

The `slice` method does for arrays what the `substring` method does for strings. It returns the part of the `Array` object specified by its parameters, which are used as subscripts. The returned array has the elements of the array object through which it is called from the first parameter up to, but not including the second parameter. For example:

```
var list = [2, 4, 6, 8, 10];
...
var list2 = list.slice(1, 3);
```

The value of `list2` is now `[4, 6]`. If `slice` is given just one parameter, the returned array has all of the elements of the object, starting with the specified index:

```
var list = ["Bill", "Will", "Jill", "dill"];
...
var listette = list.slice(2);
```

The value of `listette` is `["Jill", "dill"]`.

When the `toString` method is called through an `Array` object, each of the elements of the object is converted (if necessary) to a string. These strings are catenated, separated by commas. So, for `Array` objects, the `toString` method behaves much like `join`.

The `push`, `pop`, `unshift`, and `shift` methods of `Array` allow the easy implementation of stacks and queues in arrays. The `pop` and `push` methods remove and add an element to the high end of an array, respectively. For example, consider the following code:

```
var list = ["Dasher", "Dancer", "Donner", "Blitzen"];
var deer = list.pop();     // deer is "Blitzen"
list.push("Blitzen");
  // This puts "Blitzen" back on list
```

The `shift` and `unshift` methods remove and add an element to the beginning of an array, respectively. For example, assume that `list` is created as previously and consider the following code:

```
var deer = list.shift();    // deer is now "Dasher"
list.unshift("Dasher");
  // This puts "Dasher" back on list
```

A two-dimensional array is implemented in JavaScript as an array of arrays. This can be done with the `new` operator or with nested array literals, as shown in the following example:

```
// nested_arrays.js
//   An example illustrate an array of arrays

// Create an array object with three arrays as its elements
var nested_array = [[2, 4, 6],
                    [1, 3, 5],
                    [10, 20, 30]
                   ];

// Display the elements of nested_list
for (var row = 0; row <= 2; row++) {
  document.write("Row ", row, ":  ");

  for (var col = 0; col <=2; col++)
    document.write(nested_array[row][col], " ");

  document.write("<br />");
}
```

Figure 4.9 shows a browser display of `nested_arrays.js`.

**Figure 4.9** Display of `nested_arrays.js`

## 4.9 Functions

JavaScript functions are similar to those of other C-based languages such as C and C++. This section describes these functions.

### 4.9.1 Fundamentals

A *function definition* consists of the function's header and a compound statement that describes its actions. This compound statement is called the *body* of the function. A function *header* consists of the reserved word `function`, the function's name, and a parenthesized list of parameters, if there are any. The parentheses are required, even if there are no parameters.

A `return` statement returns control from the function in which it appears to the function's caller. Optionally, it includes an expression, whose value is returned by the function. A function body may include one or more `return` statements. If there are no `return` statements in a function, or if the specific `return` that is executed does not include an expression, the returned value is `undefined`. This is also the case if execution reaches the end of the function body without executing a `return` statement.

Syntactically, a call to a function with no parameters is the function's name followed by an empty pair of parentheses. A call to a function that returns `undefined` is a standalone statement. A call to a function that returns a useful value appears as the operand in an expression (often the whole right side of an assignment statement). For example, if `fun1` is a parameterless function that returns `undefined`, and if `fun2`, which also has no parameters, returns a useful value, they can be called with the following code:

```
fun1();
result = fun2();
```

JavaScript functions are objects, so variables that reference them can be treated as other object references. They can be passed as parameters, assigned to other variables, and can be the elements of an array. Consider the following example:

```
function fun() { document.write(
                   "This surely is fun! <br/>");}
ref_fun = fun;     // Now, ref_fun refers to the fun object
fun();             // A call to fun
ref_fun();         // Also a call to fun
```

Because JavaScript functions are objects, their addresses can be properties in other objects, in which case they act as methods.

To ensure that the interpreter sees the definition of a function before it sees a call to the function, which is required in JavaScript, function definitions are placed in the head of an XHTML document. Normally, but not always, calls to functions appear in the document body.

### 4.9.2 Local Variables

The *scope* of a variable is the range of statements over which it is visible. When JavaScript is embedded in an XHTML document, the scope of a variable is the range of lines of the document over which the variable is visible.

A variable that is not declared with a `var` statement is implicitly declared by the JavaScript interpreter at the time it is first encountered in the script. Variables that are implicitly declared, even if the implicit declaration occurs within a function definition, have *global scope*—that is, they are visible in the entire XHTML document (or entire file if the script is in its own file). Variables that are explicitly declared outside function definitions also have global scope. As stated earlier, we recommend that all variables be explicitly declared.

It is usually best for variables that are used only within a function to have *local scope*, meaning that they are visible and can be used only within the body of the function. Any variable explicitly declared with `var` in the body of a function has local scope.

If a variable that is defined both as a local variable and as a global variable appears in a function, the local variable has precedence, effectively hiding the global variable with the same name. This is the advantage of local variables: When you make up their names, you need not be concerned that a global variable with the same name may exist somewhere in the collection of scripts in the XHTML document.

Although JavaScript function definitions can be nested, the need for nested functions in client-side JavaScript is minimal. Furthermore, they can greatly complicate scripts. Therefore, we do not recommend the use of nested functions and do not discuss them.

### 4.9.3 Parameters

The parameter values that appear in a call to a function are called *actual parameters*. The parameter names that appear in the header of a function definition, which correspond to the actual parameters in calls to the function, are called

*formal parameters*. JavaScript uses the pass-by-value parameter-passing method. When a function is called, the values of the actual parameters specified in the call are, in effect, copied into their corresponding formal parameters, which behave exactly like local variables. Because references are passed as the actual parameters for objects, the function has access to the objects and can change them, thereby providing the semantics of pass-by-reference parameters. However, if a reference to an object is passed to a function and the function changes its corresponding formal parameter (rather than the object to which it points), it has no affect on the actual parameter. For example, suppose an array is passed as a parameter to a function, as in the following:

```
function fun1(my_list) {
  var list2 = new Array(1, 3, 5);
  my_list[3] = 14;
  ...
  my_list = list2;
  ...
}
...
var list = new Array(2, 4, 6, 8)
fun1(list);
```

The first assignment to `my_list` in `fun1` changes the object to which `my_list` refers, which was created in the calling code. However, the second assignment to `my_list` changes it to refer to a different array object. This does not change the actual parameter in the caller.

Because of JavaScript's dynamic typing, there is no type checking of parameters. The called function can itself check the types of parameters with the `typeof` operator. However, recall that `typeof` cannot distinguish between different objects. The number of parameters in a function call is not checked against the number of formal parameters in the called function. In the function, excess actual parameters that are passed are ignored; excess formal parameters are set to `undefined`.

All parameters are communicated through a property array, `arguments`, which, like other array objects, has a property named `length`. By accessing `arguments.length`, a function can determine the number of actual parameters that were passed. Because the `arguments` array is accessible directly, all actual parameters specified in the call are available, including actual parameters that do not correspond to any formal parameters (because there were more actual parameters than formal parameters). Consider the following example:

```
// params.js
//   The params function and a test driver for it.
//   This example illustrates a variable number of
//   function parameters
```

```
// Function params
// Parameters: A variable number of parameters
// Returns: nothing
// Displays its parameters
function params(a, b) {
  document.write("Function params was passed ",
      arguments.length, " parameter(s) <br />");
  document.write("Parameter values are: <br />");

  for (var arg = 0; arg < arguments.length; arg++)
    document.write(arguments[arg], "<br />");

  document.write("<br />");
}

// A test driver for function params
params("Mozart", "Beethoven");
params("Mozart", "Beethoven", "Tchaikowsky");
```

Figure 4.10 shows a browser display of `params.js`.

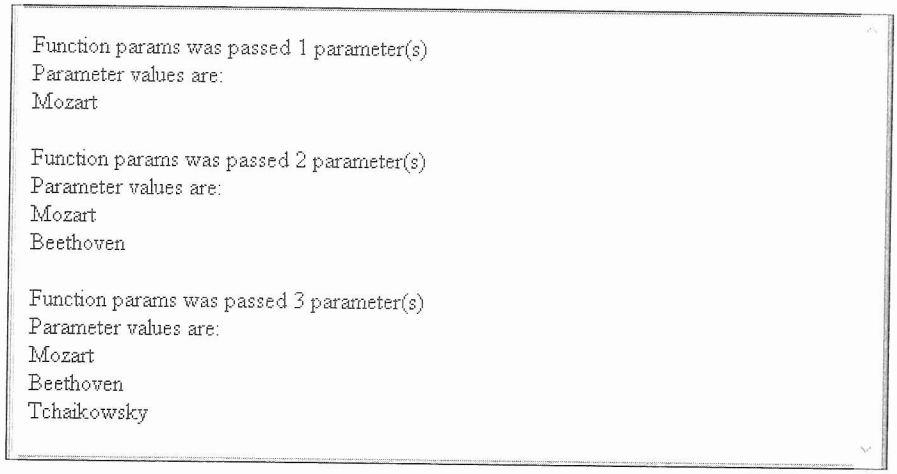


**Figure 4.10** Display of `params.js`

There is no elegant way in JavaScript to pass a primitive value by reference. One inelegant way is to put the value in an array and pass the array. This works because arrays are objects. For example, consider the following script:

```
// Function by10
//    Parameter: a number, passed as the first element
//               of an array
```

```
//  Returns: nothing
//  Effect: multiplies the parameter by 10

function by10(a) {
    a[0] *= 10;
}
...
var x;
var listx = new Array(1);
...
listx[0] = x;
by10(listx);
x = listx[0];
```

Another way to have a function change the value of a primitive type actual parameter is to have the function return the new value as follows:

```
function by10_2(a) {
    return 10 * a;
}
...
var x;
...
x = by10_2(x);
```

### 4.9.4 The sort Method, Revisited

Recall that the `sort` method for array objects converts the array's elements to strings, if necessary, and then sorts them alphabetically. If you need to sort something other than strings, or if you want an array to be sorted in some order other than alphabetically as strings, the comparison operation must be supplied to the `sort` method by the caller. Such a comparison operation is passed as a parameter to `sort`. The comparison function must return a negative number if the two elements being compared are in the desired order, zero if they are equal, and a number greater than zero if they must be interchanged. For numbers, simply subtracting the second from the first produces the required result. For example, if you want to sort the array of numbers `num_list` into descending order using the `sort` method, you could use the following:

```
// Function num_order
// Parameter: Two numbers
// Returns: If the first parameter belongs before the
//            second in descending order, a negative number
//          If the two parameters are equal, 0
//          If the two parameters must be
//            interchanged, a positive number
function num_order(a, b) {return b - a;}
```

```
// Sort the array of numbers, list, into
// ascending order
  num_list.sort(num_order);
```

Rather than defining a comparison function elsewhere and passing its name, the function definition can appear as the actual parameter in the call to `sort`. This is shown in the script in Section 4.10.

## 4.10 An Example

The following is an example of an XHTML document containing a JavaScript function to compute the median of an array of numbers. The function first sorts the array using the `sort` method. If the given array has an odd length, the median is the middle element. The middle element is determined by dividing the length by 2 and truncating the result using `floor`. If the length is even, the median is the average of the two middle elements. The result of the average computation is rounded to an integer using `round`.

```
// medians.js
//   A function and a function tester
//   Illustrates array operations

// Function median
//   Parameter: An array of numbers
//   Result: The median of the array
//   Return value: none
function median(list) {
  list.sort(function (a, b) {return a - b;});
  var list_len = list.length;

// Use the modulus operator to determine whether
//  the array's length is odd or even
// Use Math.floor to truncate numbers
// Use Math.round to round numbers
  if ((list_len % 2) == 1)
    return list[Math.floor(list_len / 2)];
  else
    return Math.round((list[list_len / 2 - 1] +
                       list[list_len / 2]) / 2);
}  // end of function median

// Test driver
var my_list_1 = [8, 3, 9, 1, 4, 7];
```

```
var my_list_2 = [10, -2, 0, 5, 3, 1, 7];
var med = median(my_list_1);
document.write("Median of [", my_list_1, "] is: ",
               med, "<br />");
med = median(my_list_2);
document.write("Median of [", my_list_2, "] is: ",
               med, "<br />");
```

Figure 4.11 shows a browser display of `medians.js`.

**Figure 4.11** Display of `medians.js`

One significant side effect of the `median` function is that it leaves the given array in ascending order. This may not always be acceptable. If not, the array could be moved to a local array in `median` before the sorting operation.

Notice that this script uses `Math.floor` to determine the median of an odd-length list. If the list subscripts began at 1, this would be wrong; because they begin at 0, it is correct.

## 4.11 Constructors

JavaScript constructors are special methods that create and initialize the properties for newly created objects. Every `new` expression must include a call to a constructor, whose name is the same as the object being created. As you saw in Section 4.8, for example, the constructor for arrays is named `Array`. Constructors are actually called by the `new` operator, which immediately precedes them in the `new` expression.

Obviously, a constructor must be able to reference the object on which it is to operate. JavaScript has a predefined reference variable for this purpose, named `this`. When the constructor is called, `this` is a reference to the newly created object. The `this` variable is used to construct and initialize the properties of the object. For example, consider the following constructor:

```
function car(new_make, new_model, new_year) {
    this.make = new_make;
    this.model = new_model;
    this.year = new_year;
}
```

This constructor could be used as in the following:

```
my_car = new car("Ford", "Contour SVT", "2000");
```

So far, we have considered only data properties. If a method is to be included in the object, it is initialized the same way as if it were a data property. For example, suppose you wanted a method for `car` objects that listed the property values. A function that could serve as such a method could be written as follows:

```
function display_car() {
    document.write("Car make: ", this.make, "<br/>");
    document.write("Car model: ", this.model, "<br/>");
    document.write("Car year: ", this.year, "<br/>");
}
```

The following line must be added to the `car` constructor:

```
this.display = display_car;
```

Now, the code `my_car.display();` would produce the following:

```
Car make: Ford
Car model: Contour SVT
Car year: 2000
```

The collection of objects created using the same constructor is related to the concept of class in an object-oriented programming language. All such objects have the same set of properties and methods, at least initially. However, there is no convenient way to determine in the script whether two objects have the same set of properties and methods.

## 4.12 Pattern Matching Using Regular Expressions

JavaScript has powerful pattern-matching capabilities based on regular expressions. There are two approaches to pattern matching in JavaScript: one that is based on the `RegExp` object and one that is based on methods of the `String` object. The regular expressions used by these two approaches are the same. They are based on the regular expressions of the Perl programing language. This book covers only the `String` methods for pattern matching.

As stated previously, patterns are specified in a form that is based on regular expressions, which were developed to define members of a simple class of formal languages. Elaborate and complex patterns can be used to describe specific strings or categories of strings. Patterns, which are sent as parameters to the pattern-matching methods, are delimited with slashes.

The simplest pattern-matching method is `search`, which takes a pattern as a parameter. The `search` method returns the position in the `String` object (through which it is called) where the pattern matched. If there is no match, `search` returns `-1`. Most characters are normal, which means that in a pattern they match themselves. For example, consider the following:

```
var str = "Rabbits are furry";
var position = str.search(/bits/);
if (position > 0)
    document.write("'bits' appears in position", position,
                   "<br />");
else
    document.write("'bits' does not appear in str <br />");
```

The output of this code is as follows:

```
'bits' appears in position 3
```

### 4.12.1 Character and Character-Class Patterns

The "normal" characters are those that are not metacharacters, which are characters that have special meanings in some contexts in patterns. The following are metacharacters:

```
\ | ( ) [ ] { } ^ $ * + ? .
```

Metacharacters can themselves be matched by being immediately preceded by a backslash.

A period matches any character except newline. So, the following pattern matches `"snowy"`, `"snowe"`, and `"snowd"`, among others:

```
/snow./
```

To match a period in a string, the pattern must backslash the period. For example, the pattern `/3\.4/` matches `3.4`. The pattern `/3.4/` would match `3.4` and `374`.

It is often convenient to be able to specify classes of characters rather than individual characters. Such classes are defined by placing the desired characters in brackets. Dashes can appear in class definitions, making it easy to specify sequences of characters. For example, you could have the following character class, which matches `'a'`, `'b'`, or `'c'`:

```
[abc]
```

Also, you could have the following character class, which matches any lowercase letter from `'a'` to `'h'`:

```
[a-h]
```

If a circumflex character (^) is the first character in a class, it inverts the specified set. For example, the following character class matches any character except the letters `'a'`, `'e'`, `'i'`, `'o'`, and `'u'`:

```
[^aeiou]
```

Because they are frequently used, some character classes are predefined and can be specified by their names. These are shown in Table 4.8, which gives the

names of the classes, their literal definitions as character classes, and descriptions of what they match.

**Table 4.8** Predefined character classes

| Name | Equivalent Pattern | Matches |
|---|---|---|
| `\d` | `[0-9]` | A digit |
| `\D` | `[^0-9]` | Not a digit |
| `\w` | `[A-Za-z_0-9]` | A word character (alphanumeric) |
| `\W` | `[^A-Za-z_0-9]` | Not a word character |
| `\s` | `[ \r\t\n\f]` | A whitespace character |
| `\S` | `[^ \r\t\n\f]` | Not a whitespace character |

Consider the following examples of patterns that use predefined character classes:

```
/\d\.\d\d/      // Matches a digit, followed by a period,
                // followed by two digits
/\D\d\D/        // Matches a single digit
/\w\w\w/        // Matches three adjacent word characters
```

In many cases, it is convenient to be able to repeat a part of a pattern, often a character or character class. To repeat a pattern, a numeric quantifier, delimited by braces, is attached. For example, the following pattern matches `xyyyyz`:

```
/xy{4}z/
```

There are also three symbolic quantifiers: asterisk (`*`), plus (`+`), and question mark (`?`). An asterisk means zero or more repetitions, a plus sign means one or more repetitions, and a question mark means one or none. For example, the following pattern matches strings that begin with any number of `x`'s (including zero), followed by one or more `y`'s, possibly followed by `z`:

```
/x*y+z?/
```

The quantifiers are often used with the predefined character class names, as in the following pattern, which matches a string of one or more digits followed by a decimal point and possibly more digits:

```
/\d+\.\d*/
```

As another example, consider the following pattern:

```
/[A-Za-z]\w*/
```

This pattern matches the identifiers in some programming languages (a letter, followed by zero or more letters, digits, or underscores).

There is one additional named pattern that is often useful. This is `\b` (boundary), which matches the boundary position between a word character (`\w`) and a nonword character (`\W`), in either order. For example, the following pattern matches `"A tulip is a flower"` but not `"A frog isn't"`:

```
/\bis\b/
```

It does not match the second string because the 'is' is followed by another word character ('n').

The boundary pattern is different from the named character classes in that it does not match a character; it matches a position between two characters.

### 4.12.2 Anchors

Frequently, it is useful to be able to specify that a pattern must match at a particular position in a string. The most common example of this is requiring a pattern to match at one specific end of the string. A pattern is tied to a string position with an anchor. A pattern can be specified to match only at the beginning of the string by preceding it with a circumflex (^) anchor. For example, the following pattern matches `"pearls are pretty"` but does not match `"My pearls are pretty"`:

```
/^pearl/
```

A pattern can be specified to match at the end of a string only by following the pattern with a dollar-sign anchor. For example, the following pattern matches `"I like gold"` but does not match `"golden"`:

```
/gold$/
```

Anchor characters are like boundary-named patterns. They do not match specific characters in the string; rather, they match positions before, between, or after characters. When a circumflex appears in a pattern at a position other than the beginning of the pattern or at the beginning of a character class, it has no special meaning (it matches itself). Likewise, if a dollar sign appears in a pattern at a position other than the end of the pattern, it has no special meaning.

### 4.12.3 Pattern Modifiers

Modifiers can be attached to patterns to change how they are used, thereby increasing their flexibility. The modifiers are specified as letters just after the right delimiter of the pattern. The `i` modifier makes the letters in the pattern match either uppercase or lowercase letters in the string. For example, the pattern `/Apple/i` matches 'APPLE', 'apple', 'APPle', and any other combination of uppercase and lowercase spellings of the word "apple."

The x modifier allows whitespace to appear in the pattern. Because comments are considered whitespace, this provides a way to include explanatory comments in the pattern. For example:

```
/\d+          # The street number
\s            # The space before the street name
[A-Z][a-z]+   # The street name
/x
```

This pattern is equivalent to the following:

```
/\d+\s[A-Z][a-z]+/
```

### 4.12.4 Other Pattern-Matching Methods of String

The replace method is used to replace substrings of the String object that match the given pattern. The replace method takes two parameters: the pattern and the replacement string. The g modifier can be attached to the pattern if the replacement is to be global in the string, in which case the replacement is done for every match in the string. The matched substrings of the string are made available through the predefined variables $1, $2, and so on. For example, consider the following statements:

```
var str = "Fred, Freddie, and Frederica were siblings";
str.replace(/Fre/g, "Boy");
```

In this example, str is set to "Boyd, Boyddie, and Boyderica were siblings", and $1, $2, and $3 are all set to "Fre".

The match method is the most general of the String pattern-matching methods. The match method takes a single parameter, a pattern. It returns an array of the results of the pattern-matching operation. If the pattern has the g modifier, the returned array has all of the substrings of the string that matched. If the pattern does not include the g modifier, the following returned array has the match as its first element, and the remainder of the array has the matches of parenthesized parts of the pattern, if there are any:

```
var str =
  "Having 4 apples is better than having 3 oranges";
var matches = str.match(/\d/g);
```

In this example, matches is set to [4, 3].

Now consider a pattern that has parenthesized subexpressions as follows:

```
var str = "I have 428 dollars, but I need 500";
var matches = str.match(/(\d+)([^\d]+)(\d+)/);
document.write(matches, "<br />");
```

The following is the value of the matches array after this code is interpreted:

```
["428 dollars, but I need 500", "428",
"dollars, but I need ", "500"]
```

In this result array, the first element is the match; the second, third, and fourth elements are the parts of the string that matched the parenthesized parts of the pattern.

The `split` method of `String` splits its object string into substrings, based on a given string or pattern. The substrings are returned in an array. For example, consider the following code:

```
var str = "grapes:apples:oranges";
var fruit = str.split(":");
```

In this example, `fruit` is set to `[grapes, apples, oranges]`.

As mentioned at the beginning of this section, there is a second way to do pattern matching in JavaScript. A pattern can be a `RegExp` object, in which case the methods of that object are used and the string on which the pattern is to be matched is sent as the parameter to the method. We do not discuss the use of `RegExp` objects for pattern matching.

## 4.13 Another Example

One of the common uses for JavaScript is to check the format of input from XHTML forms, which is discussed in detail in Chapter 5. The example in this section illustrates the use of a simple function to check a given string that is supposed to contain a phone number to determine whether its format is correct. The function uses a simple pattern match to check the phone number.

```
// forms_check.js
//   A function tst_phone_num is defined and tested.
//   This function checks the validity of phone
//   number input from a form

// Function tst_phone_num
//   Parameter: A string
//   Result: Returns true if the parameter has the form of a legal
//           seven-digit phone number (3 digits, a dash, 4 digits)

function tst_phone_num(num) {

// Use a simple pattern to check the number of digits and the dash
  var ok = num.search(/\d{3}-\d{4}/);

  if (ok == 0)
    return true;
  else
    return false;
```

```
} // end of function tst_phone_num

// A script to test tst_phone_num
var tst = tst_phone_num("444-5432");
if (tst)
  document.write("444-5432 is a legal phone number <br />");
else
  document.write("Error in tst_phone_num <br />");

tst = tst_phone_num("444-r432");
if (tst)
  document.write("Program error <br />");
else
  document.write(
            "444-r432 is not a legal phone number <br />");

tst = tst_phone_num("44-1234");
if (tst)
  document.write("Program error <br />");
else
  document.write("44-1234 is not a legal phone number <br /");
```

Figure 4.12 shows a browser display of `forms_check.js`.

444-5432 is a legal phone number
444-r432 is not a legal phone number
44-1234 is not a legal phone number

**Figure 4.12** Display of `forms_check.js`

## 4.14 Errors in Scripts

The JavaScript interpreter is capable of detecting various errors in scripts. These are primarily syntax errors, although uses of undefined variables are also detected. Debugging a script is a bit different from debugging a program in a more typical programming language, mostly because errors that are detected by the JavaScript interpreter are found while the browser is attempting to display a document. In most cases, script errors cause the browser to not display the document and do not produce an error message. Without a diagnostic message, you must simply examine the code to find the problem. This is, of course, unacceptable for all but the smallest and simplest scripts. Fortunately, there are ways to get some assistance.

The default settings for IE7 provide no debugging help for JavaScript. However, this can be changed as follows: Select *Internet Options* from the *Tools* menu and choose the *Advanced* tab there. This opens a window with a long list of checkboxes. Uncheck the *Disable script debugging* box and check the *Display a notification about every script error* box. Then press the *Apply* button in this window. Starting then and continuing until the browser is closed, JavaScript errors will cause the browser to open and display a small window with an explanation of the problem. For example, consider the following sample XHTML document:

```
// debugdemo.js
//   An example to illustrate debugging help

var row;
row = 0;

while(row != 4 {
  document.write("row is ", row, "<br />");
  row++;
}
```

Notice the syntax error in the `while` statement. Figure 4.13 shows the browser display of what happens when an attempt is made to display `debugdemo.js`.

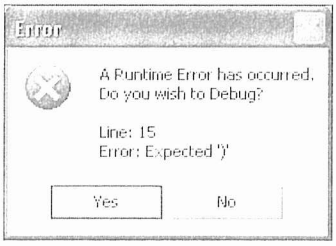


**Figure 4.13** Display of `debugdemo.js` with Internet Explorer 7

The FX2 browser has a special console window that displays script errors. Select *Tools and Error Console* to open this window. When using this browser to display documents that include JavaScript, this window should be kept open. After an error message has appeared and has been used to fix a script, press the *Clear* button on the console. Otherwise, the old error message will remain there and possibly cause confusion about subsequent problems. An example of the FX2 JavaScript Console window is shown in Figure 4.14.

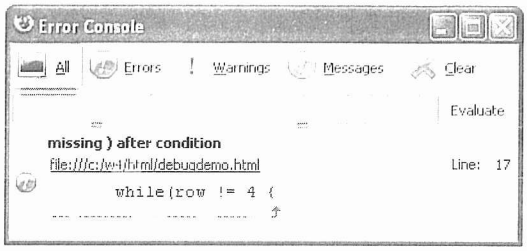


**Figure 4.14** Display of the FireFox 2 error console

The more interesting and challenging programming problems are only detectable during execution or interpretation. For these problems, a debugger is used. Both IE7 and FX2 have debuggers for JavaScript.

The IE7 JavaScript debugger is part of the browser. Documentation on it can be found at `http://msdn.microsoft.com/library/en-us/sdbug/html/sdbug_1.asp`.

The JavaScript debugger for FX2, which was produced by Mozilla and is named Venkman, is available at `http://www.mozilla.org/projects/venkman/`. Another JavaScript debugger, named Firebug, is available for FX2 at `https://addons.mozilla.org/en-US/firefox/addon/1843`. Firebug also includes a CSS debugger.

## Summary

Client-side JavaScript scripts are embedded in XHTML files as the content of `<script>` tags. The `type` attribute of `<script>` must be set to `"text/javascript"`. A file containing a script can be included by specifying its name as the value of the `<script>` attribute `src`. The script itself must appear in a special XHTML comment.

Values in JavaScript are either primitives or objects. The primitive types are Number, String, Boolean, Undefined, and Null. Numbers are represented in double-precision floating-point format. The Number, String, and Boolean types have corresponding objects named `Number`, `String`, and `Boolean`, which act as wrapper objects. String literals can use either single or double quotes as delimiters.

JavaScript is dynamically typed, which is not the same as being a typeless language. Variables are typeless, but the values they reference are typed. The type of the value referenced by a variable can change every time a new value is assigned to the variable. It is best to declare all variables explicitly.

The `Number` object includes a collection of useful properties such as `MIN_VALUE` and `PI`. The `Math` object has many methods for commonly used operations on numbers, such as `round` and `cos`. The catenation operator, `+`, creates a new string by putting the two operand strings together. The `String`

property `length` stores the number of characters in a string. There are `String` methods to return the character at a specified position in the string, the position of a specified character in the string, and a specified substring of the string. There are a large number of other `String` methods.

The `typeof` operator returns the type name of its operand if the operand is a primitive type; otherwise, it returns `"object"`.

The `Date` object provides the current time and date. It includes a large number of methods to produce various parts of time and date, such as the day of the week and the hour of the day.

The `alert` method of `Window` produces output in a dialog box. The `confirm` method of `Window` asks the user to select either an *OK* button or a *Cancel* button. The `prompt` method of `Window` asks the user for textual input. The `document.write` method dynamically produces XHTML content. The control statements of JavaScript are closely related to those of other common programming languages. Included is a `switch` statement.

Arrays in JavaScript are objects, as they are in Java. They have dynamic length. An `Array` object can be created in a `new` expression, which includes a call to the `Array` constructor, or by simply assigning an `Array` literal to a variable. `Array` literals are lists of values enclosed in brackets. Every `Array` object has a `length` property, which is both readable and writable. The property stores the number of elements in the array. `Array` objects have a collection of useful methods, among which are `join`, for joining the elements of an array in a string; `reverse`, which reverses the order of elements in an array; `sort`, which converts the elements of the array to strings and sorts them alphabetically; and `slice`, which returns a specified part of the array. The array methods `pop`, `push`, `shift`, and `unshift` were designed to implement stacks and queues in arrays.

Function definitions name their formal parameters, but do not include type names. All functions return values, but the type of the value is not specified in the function's definition. Variables declared in a function with `var` are local to that function. Parameters are passed by value, resulting in pass-by-value semantics for primitives and pass-by-reference semantics for objects. The `arguments` property stores the values of the passed parameters. Neither the types of the parameters nor the number of parameters are checked by the JavaScript interpreter.

The regular expressions used in the pattern-matching facilities of JavaScript are modeled on the regular expressions of Perl. Pattern matches are specified by one of the three methods—`search`, `replace`, or `match`—of the `String` object. The regular expressions, or patterns, are made up of special characters, normal characters, character classes, and operators. Patterns are delimited with slashes. Character classes are delimited with brackets. If a circumflex appears at the left end of a character class, it inverts the meaning of the characters in the class. Several of the most common character classes are predefined. Subpatterns can be followed by numeric or symbolic quantifiers. Patterns can be anchored at the left or right end of the string against which the pattern is being matched. The `search` method searches its object string for the pattern given as its parameter. The `replace` method replaces matches in its object string with its

second parameter. The `match` method searches its object string for the given pattern and returns an array of all matches.

## Review Questions

4.1 Describe briefly three major differences between Java and JavaScript.

4.2 Describe briefly three major uses of JavaScript on the client side.

4.3 Describe briefly the basic process of event-driven computation.

4.4 What are the two categories of properties in JavaScript?

4.5 Why does JavaScript have two categories of data variables, primitives and objects?

4.6 Describe the two ways to embed a JavaScript script in an XHTML document.

4.7 What are the two forms of JavaScript comments?

4.8 Why are JavaScript scripts sometimes hidden in XHTML documents by putting them in XHTML comments?

4.9 What are the five primitive data types in JavaScript?

4.10 Do single-quoted string literals have any different characteristics than double-quoted string literals?

4.11 In what circumstances would a variable have the value `undefined`?

4.12 If the value `undefined` is used as a Boolean expression, is it interpreted as `true` or `false`?

4.13 What purpose do rules of operator precedence serve in a programming language?

4.14 What purpose do rules of operator associativity serve in a programming language?

4.15 Describe the purpose and characteristics of `NaN`.

4.16 Why is `parseInt` not used more often?

4.17 What value does `typeof` return for an object operand?

4.18 What is the usual end-of-line punctuation for the string operand to `document.write`?

4.19 What is the usual end-of-line punctuation for the string operand to `alert`?

4.20 Describe the operation of the `prompt` method.

4.21 What is a control construct?

4.22 Must the `then` clause of an `if` statement in JavaScript always be a compound statement?

4.23 What are the three possible forms of control expressions in JavaScript?

4.24 What is the difference between `==` and `===`?

4.25 Explain what short-circuit evaluation of an expression means.

4.26 What are the semantics of a `break` statement?

4.27 What is the difference between a `while` statement and a `do-while` statement?

4.28 When is a JavaScript constructor called?

4.29 What is the difference between a constructor in Java and one in JavaScript?

4.30 What properties does an object created with a `new` operator and the `Object` constructor have?

4.31 Describe the two ways the properties of an object can be referenced.

4.32 How is a new property of an object created?

4.33 Describe the semantics of the `for-in` statement.

4.34 Describe the two ways an `Array` object can be created.

4.35 What relationship is there between the value of the `length` property of an `Array` object and the actual number of existing elements in the object?

4.36 Describe the semantics of the `join` method of `Array`.

4.37 Describe the semantics of the `slice` method when it is given just one parameter.

4.38 What is the form of a nested array literal?

4.39 What value is returned by a function that contains no `return` statement?

4.40 Define the scope of a variable in a JavaScript script embedded in an XHTML document when the variable is not declared in a function.

4.41 Is it possible to reference global variables in a JavaScript function?

4.42 What is the advantage of using local variables in functions?

4.43 What parameter-passing method does JavaScript use?

4.44 Does JavaScript check the types of actual parameters against the types of their corresponding formal parameters?

4.45 How can a function access actual parameter values for those actual parameters that do not correspond to any formal parameter?

4.46 What is one way in which primitive variables can be passed by reference to a function?

4.47 In JavaScript, what exactly does a constructor do?

4.48 What is a character class in a pattern?

4.49 What are the predefined character classes, and what do they mean?

4.50 What are the symbolic quantifiers, and what do they mean?

4.51 Describe the two end-of-line anchors.

4.52 What does the `i` pattern modifier do?

4.53 What exactly does the `String` method `replace` do?

4.54 What exactly does the `String` method `match` do?

## Exercises

Write, test, and debug (if necessary) XHTML files and JavaScript scripts for the following problems. When required to write functions, you must include a script to test the function with at least two different data sets.

4.1 *Output*: A table of the numbers from 5 to 15 and their squares and cubes, using `alert`.

4.2 *Output*: The first 20 Fibonacci numbers, which are defined as in the following sequence

1, 1, 2, 3, ...

where each number in the sequence after the second is the sum of the two previous numbers. You must use `document.write` to produce the output.

4.3 *Input*: Three numbers, using `prompt` to get each.

*Output*: The largest of the three input numbers.

*Hint*: Use the predefined function `Math.max`.

4.4 Modify the script of Exercise 4.2 to input a number, `n`, using `prompt`, which is the number of the Fibonacci number required as output.

4.5 *Input*: A text string, using `prompt`.

*Output*: Either `"Legal name"` or `"Illegal name"`, depending on whether the input names fit the required format, which is

```
Last name, first name, middle initial
```

where neither of the names can have more than 15 characters.

4.6 *Input*: A line of text, using `prompt`.

*Output*: The words of the input text, in alphabetical order.

4.7 Modify the script for Exercise 4.6 to get a second input from the user, which is either `"ascending"` or `"descending"`. Use this input to determine how to sort the input words.

4.8 *Function*: `no_zeros`

*Parameter*: An array of numbers.

*Result*: The given array must be modified to remove all zero values.

*Returns*: `true` if the given array included zero values; `false` otherwise.

4.9 *Function*: `e_names`

*Parameter*: An array of names, represented as strings.

*Returns*: The number of names in the given array that end in either `"ie"` or `"y"`.

4.10 *Function*: `first_vowel`

*Parameter*: A string.

*Returns*: The position in the string of the leftmost vowel.

4.11 *Function*: `counter`

*Parameter*: An array of numbers.

*Returns*: The numbers of negative elements, zeros, and values greater than zero in the given array.

*Note*: You must use a `switch` statement in the function.

4.12 *Function*: `tst_name`

*Parameter*: A string.

*Returns*: `true` if the given string has the form

```
string1, string2 letter
```

where both strings must be all lowercase letters except the first letter, and `letter` must be uppercase; `false` otherwise.

4.13 *Function*: `row_averages`

*Parameter*: An array of arrays of numbers.

*Returns*: An array of the averages of each of the rows of the given matrix.

4.14 *Function*: `reverser`

*Parameter*: A number.

*Returns*: The number with its digits in reverse order.

CHAPTER

5

# JavaScript and XHTML Documents


5.1 The JavaScript Execution Environment
5.2 The Document Object Model
5.3 Element Access in JavaScript
5.4 Events and Event Handling
5.5 Handling Events from Body Elements
5.6 Handling Events from Button Elements
5.7 Handling Events from Text Box and Password Elements
5.8 The DOM 2 Event Model
5.9 The `navigator` Object
5.10 DOM Tree Traversal and Modification
*Summary • Review Questions • Exercises*


**Client-side JavaScript does not include language constructs that are not in core JavaScript.** Instead, it defines the collection of objects, methods, and properties that allow scripts to interact with XHTML documents on the client. This chapter describes some of these features and illustrates their use with examples.

The chapter begins with a description of the execution environment of client-side JavaScript, which means the object hierarchy that corresponds to the structure of documents. Then it gives a brief overview of the Document Object Model (DOM), noting that you need not know the details of this model to be able to use client-side JavaScript. Next, the techniques for accessing XHTML

document elements in JavaScript are discussed. The fundamental concepts of events and event handling are then introduced, using the basic event model. Although the event-driven model of computation is not a new idea in programming, it has become more important to programmers with the advent of Web programming. Next, the chapter describes the relationships between event objects, XHTML tag attributes, and tags, primarily by means of two tables.

Applications of basic event handling are introduced through a sequence of complete document/JavaScript examples. The first of these illustrates handling the `load` event from a body element. The next two examples demonstrate the use of the `click` event created when radio buttons are pressed. This is followed by an example that uses the `blur` event to compare passwords that are input twice. The next example demonstrates the use of the `change` event to validate the format of input to a text box. The last example shows the use of the `blur` event to prevent user changes to the values of text box elements.

Next, the current standard event model, DOM 2, is introduced, using a revision of an earlier example to illustrate the new features of this model. Finally, the chapter discusses the use of the `navigator` object to determine which browser is being used.

Nearly all of the JavaScript in the examples in this chapter is in separate files. Therefore, each of the examples consists of an XHTML document and one or two JavaScript files.

## 5.1 The JavaScript Execution Environment

A browser displays an XHTML document in a window on the screen of the client. The JavaScript `Window` object represents the window that displays the document.

All JavaScript variables are properties of some object. The properties of the `Window` object are visible to all JavaScript scripts that appear in the window's XHTML document, so they include all of the global variables. When a global variable is implicitly created in a client-side script, it is created as a new property of the `Window` object. The `Window` object provides the largest enclosing referencing environment for JavaScript scripts.

There can be more than one `Window` object. In this book, however, we deal only with scripts with a single `Window` object.

The JavaScript `Document` object represents the displayed XHTML document. Every `Window` object has a property named `document`, which is a reference to the `Document` object that the window displays. Because a `Window` object can include multiple frames, it has a property array, `frames`, whose elements are references to the frames.

Every `Document` object has a `forms` array, each element of which represents a form in the document. Each `Forms` array element has an `elements` array as a property, which contains the objects that represent the XHTML form elements, such as buttons and menus. The JavaScript objects associated with the

elements in a document can be addressed in a script in several ways. These are discussed in Section 5.3.

`Document` objects also have property arrays for anchors, links, images, and applets. There are many other objects in the object hierarchy below a `Window` object, but in this chapter we are primarily interested in documents, forms, and form elements.

## 5.2 The Document Object Model

The Document Object Model (DOM) has been under development by the W3C since the mid-1990s. At the time of this writing, DOM Level 2 (usually referred to as DOM 2) was the latest approved version, and DOM 3 was under development. The original motivation for the standard DOM was to provide a specification that would make Java programs and JavaScript scripts that deal with XHTML documents portable among various browsers.

Although the W3C never produced such a specification, DOM 0 is the name often used to describe the document model used by the early browsers that supported JavaScript. Specifically, DOM 0 is the version of the document model implemented in the Netscape 3.0 and Internet Explorer 3.0 browsers. The DOM 0 model was partially documented in the HTML 4 specification.

DOM 1, the first W3C DOM specification, issued in October 1998, focused on the XHTML and XML (see Chapter 7, "Introduction to XML") document model. DOM 2, issued in November 2000, specifies a style sheet object model and defines how style information attached to a document can be manipulated. It also includes document traversals and provides a complete and comprehensive event model. DOM 3 will deal with content models for XML (DTDs and schemas), document validation, and document views and formatting, as well as key events and event groups. As stated previously, DOM 0 is supported by all JavaScript-enabled browsers. DOM 2 is nearly completely supported by Firefox 2 (FX2), but Internet Explorer 7 (IE7) leaves significant parts either unimplemented or implemented in a nonstandard way.

The DOM is an application programming interface (API) that defines an interface between XHTML documents and application programs. It is an abstract model because it must apply to a variety of application programming languages. Each language that interfaces with the DOM must define a binding to that interface. The actual DOM specification consists of a collection of interfaces, including one for each document tree node type. These interfaces are similar to Java interfaces and C++ abstract classes. They define the objects, methods, and properties that are associated with their respective node types. With the DOM, users can write code in programming languages to create documents, move around in their structures, and change, add, or delete elements and their content.

Documents in the DOM have a treelike structure, but there can be more than one tree in a document (though that is unusual). Because the DOM is an abstract interface, it does not dictate that documents must be implemented as

trees or collections of trees. Therefore, in an implementation, the relationships among the elements of a document can be represented in any number of different ways.

The following XHTML document and its corresponding DOM tree illustrate the relationship between them.

```
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> A simple document </title>
  </head>
  <body>
    <table>
      <tr>
        <th> Breakfast </th>
        <td> 0 </td>
        <td> 1 </td>
      </tr>
      <tr>
        <th> Lunch </th>
        <td> 1 </td>
        <td> 0 </td>
      </tr>
    </table>
  </body>
</html>
```

Figure 5.1 shows the DOM structure for this table.

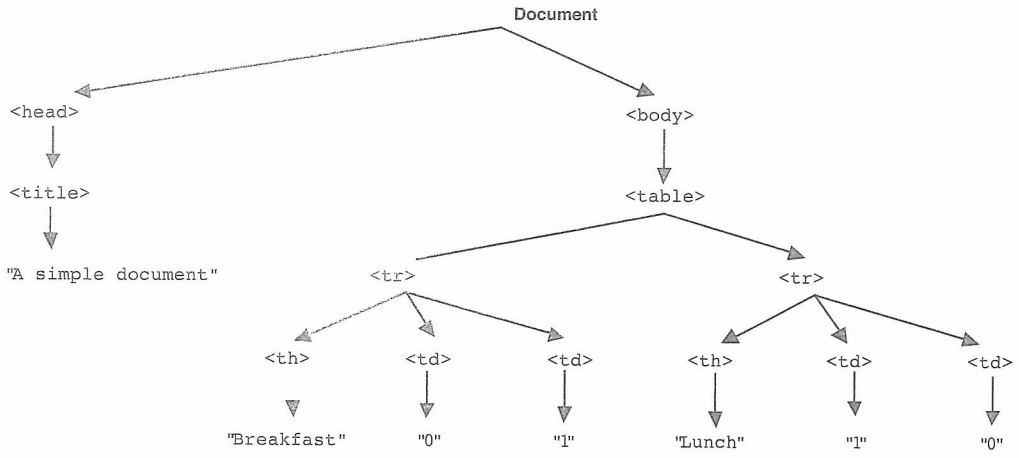


**Figure 5.1** The DOM structure for a simple document

A language that is designed to support the DOM must have a binding to the DOM constructs. This binding amounts to a correspondence between constructs in the language and elements in the DOM. In the JavaScript binding to

the DOM, the elements of a document are objects, with both data and operations. The data are called *properties*, and the operations are, naturally, called *methods*. For example, the following element would be represented as an object with two properties, `type` and `name`, with the values `"text"` and `"address"`, respectively:

```
<input type = "text"  name = "address">
```

In most cases, the property names in JavaScript are the same as their corresponding attribute names in XHTML.

Anything resembling a complete explanation of the DOM is far beyond the scope of this book. Therefore, our introduction to the DOM here is intended only to provide the basis for our discussion of how JavaScript can be used to respond to document-related events and dynamically modify element attributes and content.[1] Specifically, we do not cover document tree traversal, adding and deleting nodes, or reordering nodes. A detailed description of the DOM can be found at the W3C Web site.

## 5.3 Element Access in JavaScript

The elements of an XHTML document have corresponding objects that are visible to an embedded JavaScript script. The addresses of these objects are required, both by the event handling discussed in this chapter and by the code to make dynamic changes to documents, which is discussed in Chapter 6, "Dynamic Documents with JavaScript."

There are several ways the object associated with an XHTML form element can be addressed in JavaScript. The original (DOM 0) way is to use the `forms` and `elements` arrays of the `Document` object, which is referenced through the `document` property of the Window object. As an example, consider the following XHTML document:

```
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Access to form elements </title>
  </head>
  <body>
    <form action = "">
      <input type = "button"  name = "turnItOn" />
    </form>
  </body>
</html>
```

We refer to the address of the JavaScript object that is associated with an XHTML element as its *DOM address*. The DOM address of the button in this example, using the `forms` and `elements` arrays, is as follows:

```
var dom = document.forms[0].elements[0];
```

1. We will discuss modifications of style properties in Chapter 6.

The problem with this approach to element addressing is that the DOM address is defined by address elements that could change; namely, the `forms` and `elements` arrays. For example, if a new button were added before the `turnItOn` button in the document, the DOM address shown would be wrong.

Another approach to DOM addressing is to use element names. For this, the element and its enclosing elements, up to but not including the body element, must include `name` attributes. For example, consider the following document:

```
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Access to form elements </title>
  </head>
  <body>
    <form name = "myForm"  action = "">
      <input type = "button"  name = "turnItOn" />
    </form>
  </body>
</html>
```

Using the `name` attributes, the button's DOM address is as follows:

```
var dom = document.myForm.turnItOn;
```

One drawback of this approach is that the XHTML 1.1 standard does not allow the `name` attribute in the form element, even though the attribute is now legal for form elements. This is a validation problem, but it causes no difficulty for browsers.

Although `name` attributes are not allowed on form elements, name attributes are often required on the elements in a form. The server-side programs and scripts that process form data use a widget's name attribute value to identify the form data value associated with the widget.

Yet another approach to element addressing is to use the JavaScript method `getElementById`, which is defined in DOM 1. Because an element's identifier (`id`) is unique in the document, this approach works, regardless of how deeply the element is nested in other elements in the document. For example, if the `id` attribute of our button is set to `"turnItOn"`, the following could be used to get the DOM address of that button element:

```
var dom = document.getElementById("turnItOn");
```

The parameter of `getElementById` can be any expression that evaluates to a string. In many cases, it is a variable.

Because `id`s are most useful for DOM addressing and `name`s are required for form-processing code, form elements often have both `id`s and `name`s, both set to the same value.

Buttons in a group of checkboxes often share the same name. The buttons in a radio button group *always* have the same name. In these cases, the names of the individual buttons obviously cannot be used in their DOM addresses. Of course, each radio button and checkbox can have an `id`, which would make it

easy to address using `getElementById`. However, using this approach does not provide a convenient way to search a group of radio buttons or checkboxes to determine which is checked.

An alternative to both `names` and `ids` is provided by the implicit arrays associated with each checkbox and radio button group. Every such group has an array, which has the same name as the group name, that stores the DOM addresses of the individual buttons in the group. These arrays are properties of the form in which the buttons appear. To access the arrays, the DOM address of the form object first must be obtained. For example:

```
<form id = "vehicleGroup">
  <input type = "checkbox"  name = "vehicles"
         value = "car" />  Car
  <input type = "checkbox"  name = "vehicles"
         value = "truck" />  Truck
  <input type = "checkbox"  name = "vehicles"
         value = "bike" />  Bike
</form>
```

The implicit array, `vehicles`, has three elements, which reference the three objects associated with the three checkbox elements in the group. This array provides a convenient way to search the list of checkboxes in a group. The `checked` property of a checkbox object is set to `true` if the button is checked. For the preceding sample checkbox group, the following code would count the number of checkboxes that were checked:

```
var numChecked = 0;
var dom = document.getElementById("vehicleGroup");
for (index = 0; index < dom.vehicles.length; index++)
  if (dom.vehicles[index].checked)
    numChecked++;
```

Radio buttons can be addressed and handled exactly as we have addressed and handled checkboxes in the above code.

## 5.4 Events and Event Handling

The HTML 4.0 standard provided the first specification of an event model for documents. This is sometimes referred to as the DOM 0 event model. Although the DOM 0 event model is limited in scope, it is the only one that is supported by all browsers that support JavaScript. A complete and comprehensive event model was specified by DOM 2. The DOM 2 model is supported by the FX2 browser. However, inexplicably, IE7 does not support it. Our discussion of events and event handling is divided into two parts, one for the DOM 0 model and one for the DOM 2 model. We describe the DOM 2 standard, even though IE7 does not support it. It is our hope that Microsoft will soon recognize the error of its ways and implement the DOM 2 event model in its browsers.

### 5.4.1 Basic Concepts of Event Handling

One important category of use of JavaScript for Web programming is to detect certain activities of the browser and the browser user and provide computation when these activities occur. These computations are specified using a special form of programming called *event-driven programming*. In conventional (non-event-driven) programming, the code itself specifies the order in which that code is executed, although the order is usually affected by the program's input data. In event-driven programming, parts of the program are executed at completely unpredictable times, often triggered by user interactions with the executing program.

An *event* is a notification that something specific has occurred, either with the browser, such as the completion of the loading of a document, or because of a browser user action, such as a mouse click on a form button. Strictly speaking, an event is an object that is implicitly created by the browser and the JavaScript system in response to something happening.

An *event handler* is a script that is implicitly executed in response to the appearance of an event. Event handlers enable a Web document to be responsive to browser and user activities. One of the most common uses of event handlers is to check for simple errors and omissions in user input to the elements of a form, either when they are changed or when the form is submitted. This saves the time of sending the form data to the server, where its correctness then must be checked by a server-resident program or script before it can be processed.

If you are familiar with the exceptions and exception-handling capabilities of a programming language such as C++ or Java, you should see the close relationship between events and exceptions. Both events and exceptions occur at unpredictable times, and both often require some specific program actions.

Because events are JavaScript objects, their names are case sensitive. The names of all event objects have only lowercase letters. For example, `click` is an event, but `Click` is not.

Events are created by activities associated with specific XHTML elements. For example, the `click` event can be caused by the browser user clicking a radio button or the link of an anchor tag, among other things. Thus, an event's name is only part of the information pertinent to handling the event. In most cases, the specific XHTML element that caused the event is also needed.

The process of connecting an event handler to an event is called *registration*. There are two distinct approaches to event handler registration, one that assigns tag attributes and one that assigns handler addresses to object properties. These are further discussed and shown in Sections 5.5 and 5.6.

The `write` method of `document` should never be used in an event handler. Remember that a document is displayed as its XHTML code is parsed by the browser. Events usually occur after the whole document is displayed. If `write` appears in an event handler, the content generated by it might be placed over the top of the existing document.

The remainder of this section and Sections 5.5 to 5.7 describe the DOM 0 event model and some of its uses.

### 5.4.2 Events, Attributes, and Tags

HTML 4 defined a collection of events, which browsers implement and with which JavaScript can deal. These events are associated with XHTML tag attributes, which can be used to connect the events to handlers. The attributes have names that are closely related to their associated events. Table 5.1 lists the most commonly used events and their associated tag attributes.

**Table 5.1** Events and their tag attributes

| Event | Tag Attribute |
|---|---|
| blur | onblur |
| change | onchange |
| click | onclick |
| dblclick | ondblclick |
| focus | onfocus |
| keydown | onkeydown |
| keypress | onkeypress |
| keyup | onkeyup |
| load | onload |
| mousedown | onmousedown |
| mousemove | onmousemove |
| mouseout | onmouseout |
| mouseover | onmouseover |
| mouseup | onmouseup |
| reset | onreset |
| select | onselect |
| submit | onsubmit |
| unload | onunload |

In many cases, the same attribute can appear in several different tags. The circumstances under which an event is created are related to a tag and an attribute, and they can be different for the same attribute when it appears in different tags.

An XHTML text element is said to *get focus* when the user puts the mouse cursor over it and clicks the left mouse button. An element can also get focus

when the user tabs to the element. Focus on an element can be forced with the `focus` method, which is described in Section 5.4.5. When a text element has focus, any keyboard input goes into that element. Obviously, only one text element can have focus at one time. An element becomes blurred when the user moves the cursor away from the element and clicks the left mouse button, or tabs away from the element. An element obviously becomes blurred when another element gets focus. Several nontext elements can also have focus, but the condition is less useful in those cases.

Table 5.2 shows the most commonly used attributes related to events, tags that can include the attributes, and the circumstances under which the associated events are created. Only a few of the situations shown in Table 5.2 are discussed in this chapter.

**Table 5.2** Event attributes and their tags

| Attribute | Tag | Description |
|---|---|---|
| onblur | <a> | The link loses the input focus |
| | <button> | The button loses the input focus |
| | <input> | The input element loses the input focus |
| | <textarea> | The text area loses the input focus |
| | <select> | The selection element loses the input focus |
| onchange | <input> | The input element is changed and loses the input focus |
| | <textarea> | The text area is changed and loses the input focus |
| | <select> | The selection element is changed and loses the input focus |
| onclick | <a> | The user clicks on the link |
| | <input> | The input element is clicked |
| ondblclick | Most elements | The user double clicks the left mouse button |
| onfocus | <a> | The link acquires the input focus |
| | <input> | The input element receives the input focus |
| | <textarea> | A text area receives the input focus |
| | <select> | A selection element receives the input focus |
| onkeydown | <body>, form elements | A key is pressed down |
| onkeypress | <body>, form elements | A key is pressed down and released |
| onkeyup | <body>, form elements | A key is released |
| onload | <body> | The document is finished loading |

**Table 5.2** Event attributes and their tags (*continued*)

| Attribute | Tag | Description |
|---|---|---|
| onmousedown | Most elements | The user clicks the left mouse button |
| onmousemove | Most elements | The user moves the mouse cursor within the element |
| onmouseout | Most elements | The mouse cursor is moved away from being over the element |
| onmouseover | Most elements | The mouse cursor is moved over the element |
| onmouseup | Most elements | The left mouse button is unclicked |
| onreset | <form> | The reset button is clicked |
| onselect | <input> | The mouse cursor is moved over the element |
| | <textarea> | The text area is selected within the text area |
| onsubmit | <form> | The *Submit* button is pressed |
| onunload | <body> | The user exits the document |

As mentioned previously, there are two ways to register an event handler in the DOM 0 event model. One of these is by assigning the event handler script to an event tag attribute, as in the following example:

```
<input type = "button" id = "myButton"
       onclick = "alert('You clicked my button!');" />
```

In many cases, the handler consists of more than a single statement. For these, often a function is used, and the literal string value of the attribute is the call to the function as follows:

```
<input type = "button" id = "myButton"
       onclick = "myButtonHandler();" />
```

The event handler could also be registered by the assignment to the associated event property on the button object as follows:

```
document.getElementById("myButton").onclick =
                                        myButtonHandler;
```

This statement must follow both the handler function and the form element so that JavaScript has seen both before assigning the property. Notice that only the name of the handler function is assigned to the property—it is neither a string nor a call to the function.

## 5.5 Handling Events from Body Elements

The events most often created by body elements are `load` and `unload`. As our first example of event handling, we consider the simple case of producing an

alert message when the body of the document has been loaded. In this case, we use the `onload` attribute of `<body>` to specify the event handler.

```
<?xml version = "1.0"  encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- load.html
     A document for load.js
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> load.html </title>
    <script type = "text/javascript"  src = "load.js" >
    </script>
  </head>
  <body onload="load_greeting();">
    <p />
  </body>
</html>
```

```
// load.js
//   An example to illustrate the load event

// The onload event handler
function load_greeting () {
  alert("You are visiting the home page of \n" +
        "Pete's Pickled Peppers \n" + "WELCOME!!!");
}
```

Figure 5.2 shows a browser display of `load.html`.

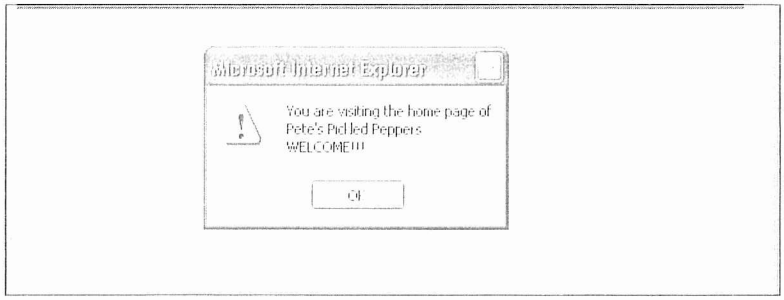


**Figure 5.2** Display of `load.html`

The `unload` event is probably more useful than the `load` event. It is used to do some cleanup before a document is unloaded, such as when the browser user goes on to some new document. For example, if the document opened a second browser window, that window should be closed by an `unload` event handler.

## 5.6 Handling Events from Button Elements

Buttons in a Web document provide a simple and effective way to collect simple input from the browser user. The most commonly used event created by button actions is `click`. Section 5.4.2 includes an example of a plain button.

Consider the following example of a set of radio buttons that enables the user to choose information about a specific airplane. The `click` event is used in this example to trigger a call to `alert`, which presents a brief description of the selected airplane. In this example, the calls to the event handlers send the value of the pressed radio button to the handler. This is another way the handler can determine which of a group of radio buttons is pressed.

```
<?xml version = "1.0" encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- radio_click.hmtl
     A document for radio_click.js
     Creates four radio buttons that call the planeChoice
     event handler to display descriptions
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> radio_click.html </title>
  <script type = "text/javascript"  src = "radio_click.js" >
  </script>
  </head>
  <body>
    <h4> Cessna single-engine airplane descriptions </h4>
    <form id = "myForm"  action = "">
      <p>
        <label> <input type = "radio"  name = "planeButton"
                       value = "152"
                       onclick = "planeChoice(152)" />
        Model 152 </label>
        <br />
        <label> <input type = "radio"  name = "planeButton"
                       value = "172"
                       onclick = "planeChoice(172)" />
```